



AMERICAN
UNIVERSITY
OF BEIRUT

Fall 2025

EECE 490: Introduction to Machine Learning

C3 - Exploratory Data Analysis

The need for Exploratory Data Analysis

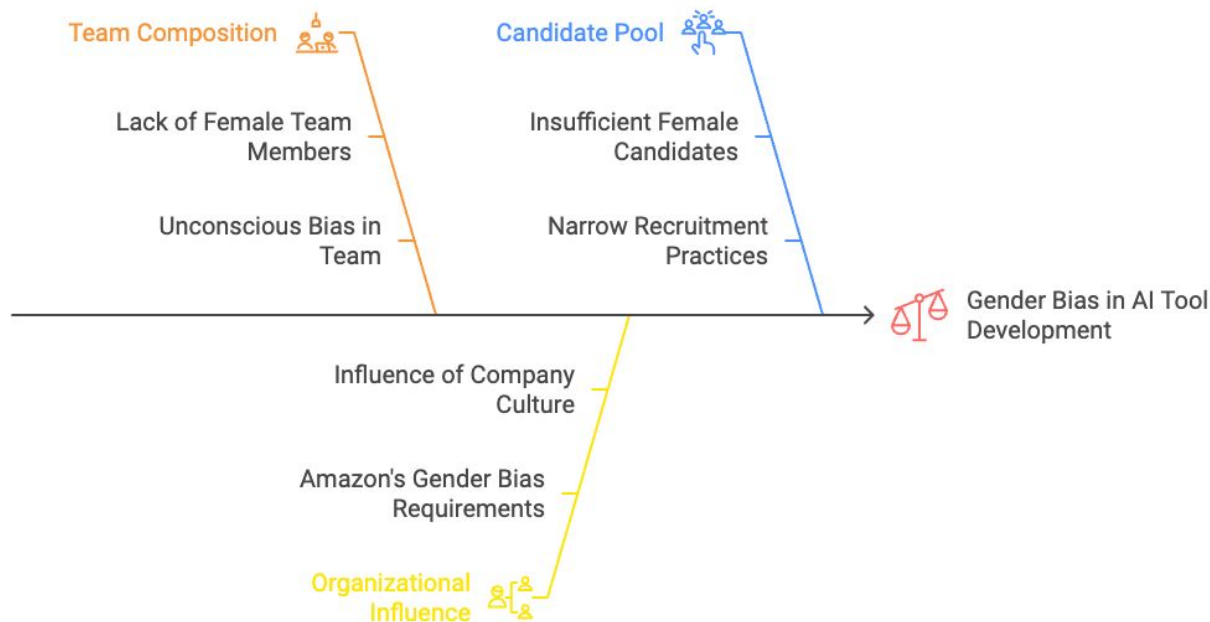
Amazon's Gender Biased HR Tool

In 2014, a team at amazon developed an **HR tool to filter out resumes**. The goal was to insert 100 resumes into the tool and get the top 5 candidates as a result.

A year later, they realized that **the tool was recommending only male candidates**. The project was later abandoned, causing Amazon to publicly claim that the tool was the reason for the low percentage of female employees.

Amazon's Gender Biased HR Tool

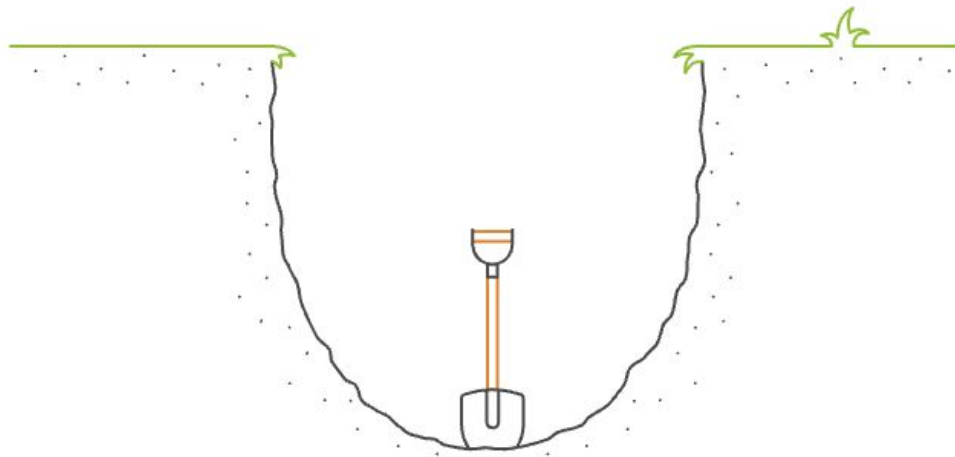
What could be the reason for the bias in the HR tool?



Exploring and Pruning Data

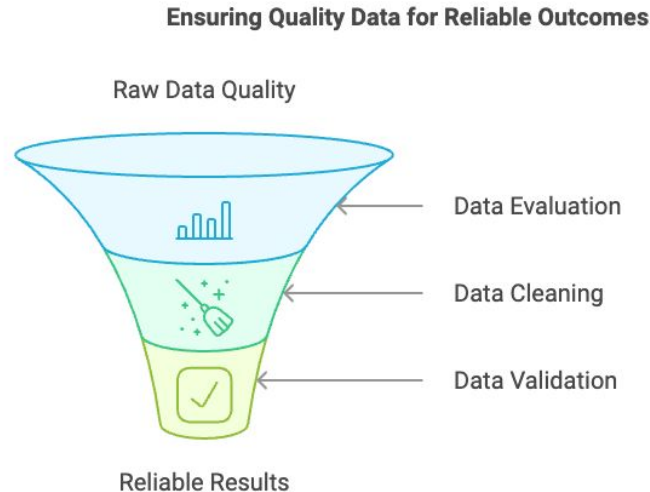
If the team at Amazon paid more attention to the **information and quality** within their **training dataset**, they would have noticed that the **Gender Feature** should not have been included as an input.

Poor data input leads to unreliable results.



Exploring and Pruning Data

Understanding all features and dimensions within your data would allow you to make the right feature cleaning and selection decisions. If you fed the model bias data, you would get biased predictions.



Garbage in - Garbage out. The first rule of ML


 + Machine Learning = 

Data


 + Artificial Intelligence = 

Data


 + Generative AI = 

Data

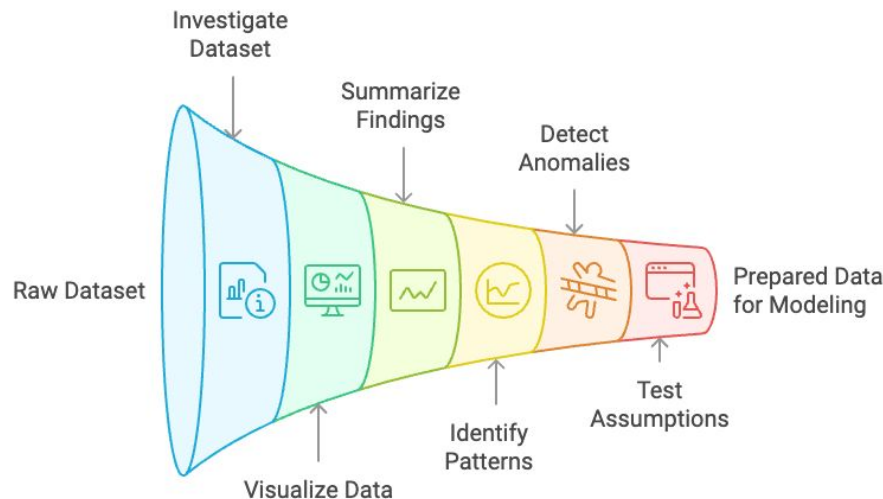

 + Agentic AI = 

Data

Where Exploratory Data Analysis Comes in

Exploratory Data Analysis (**EDA**) is the process of **investigating**, **visualizing**, and **summarizing** a dataset to uncover its underlying **structure**, **identify patterns**, **detect anomalies**, and **test assumptions**—all with the aim of preparing the data for further analysis or modeling.

EDA Process for Data Preparation



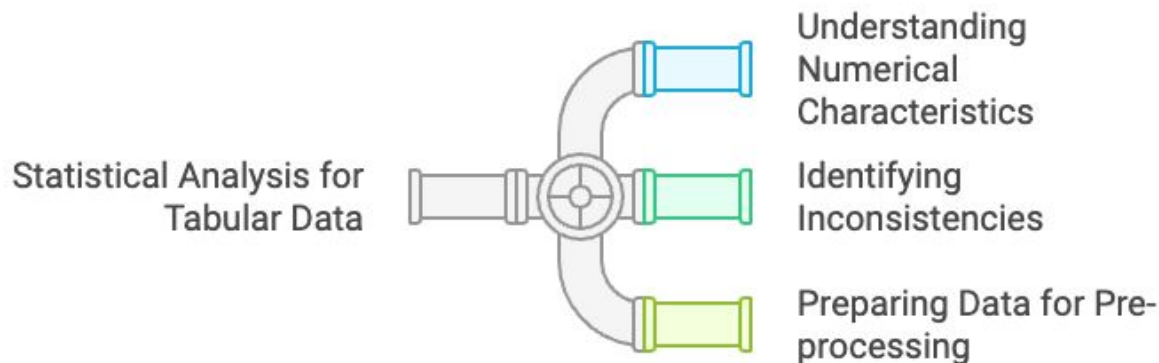
Understanding Your Data

There are many dimensions to explore and understand your data before cleaning and transforming it for the machine learning model. These dimensions include:

1. Statistical Analysis:
 - a. Distribution Analysis
 - b. Outlier Detection
2. Visualization Analysis
3. Dimensionality Exploration

Statistical Analysis: Definition

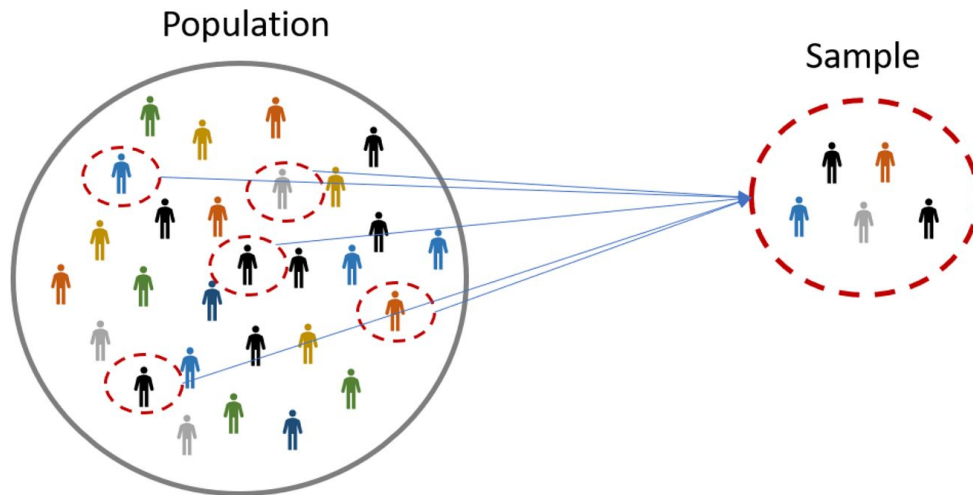
Statistical analysis for tabular data focuses on **understanding the dataset's underlying numerical characteristics, identifying inconsistencies, and preparing the data for pre-processing.**



Statistical Analysis: Distribution Analysis

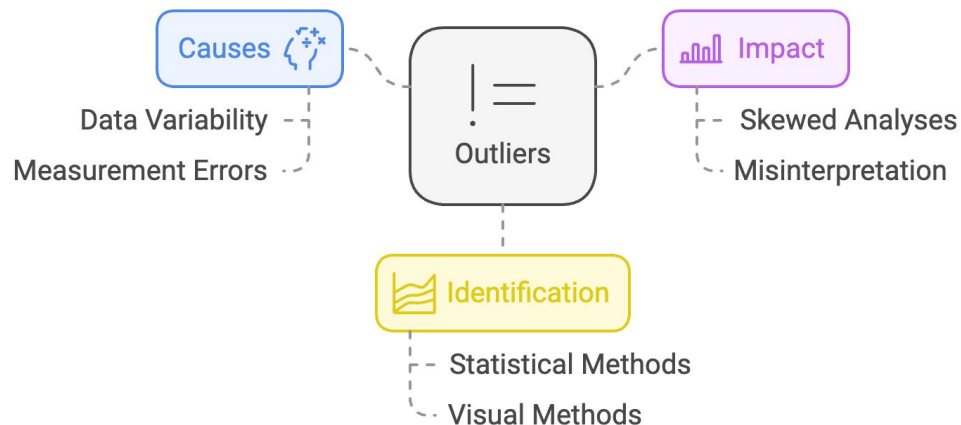
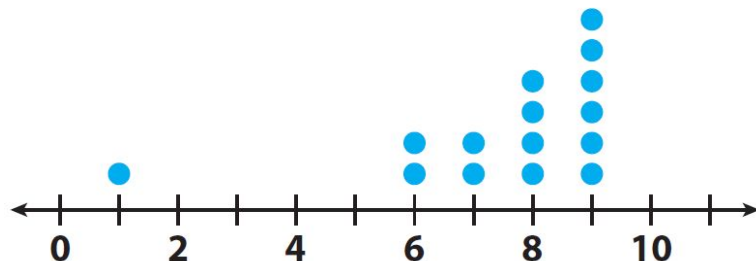
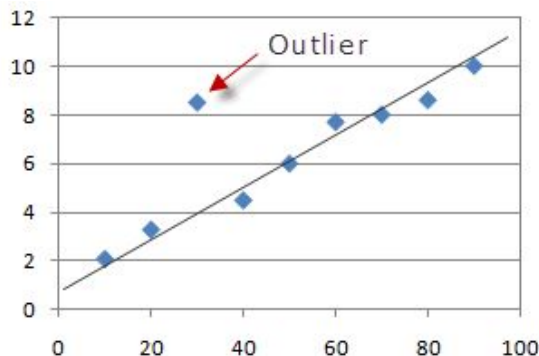
Definition: Analyzing the **spread and shape** of data values to uncover **trends**, **skewness**, and **variability**.

A dataset is always taken as a sample from a population. We need to understand if this population is **fairly represented**.



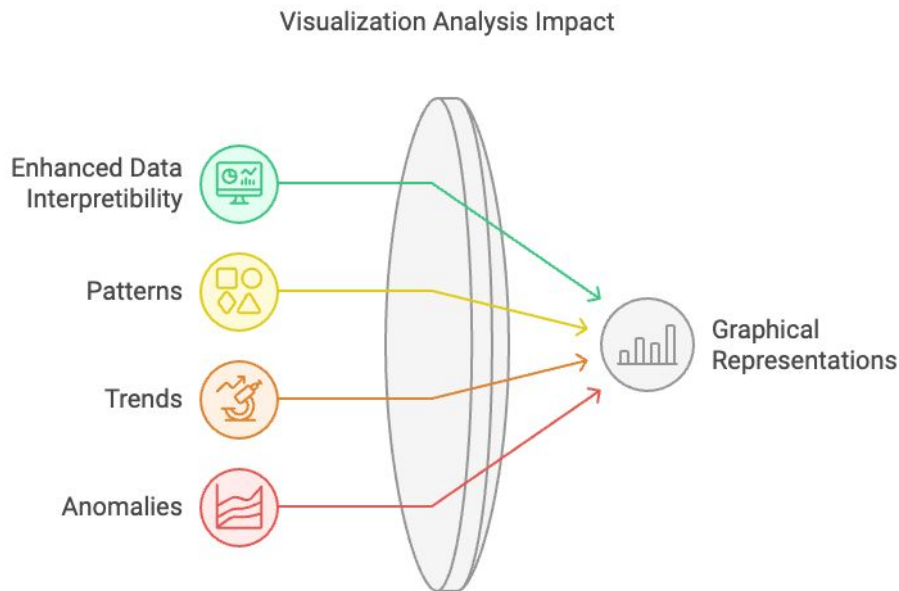
Statistical Analysis: Outlier Detection

An Outlier is a data point that differs significantly from other observations.



Visualization Analysis: Definition

Using graphical representations to uncover patterns, trends, and anomalies, making data more interpretable and actionable.



Dimensionality Exploration: Definition

Dataset Dimensionality: defined by the features/samples ratio.

If number of features $\gg$ number of samples, data is high-dimensional.

Vice versa applies

Exploring dimensionality of datasets to understand structure, clustering, and feature importance effectively.

Determine Feature Importance

Ascertain which features are most influential in the dataset.

Identify Clusters

Discover groups or patterns within the data that share similarities.

Understand Structure

Grasp the foundational layout and organization of the dataset.



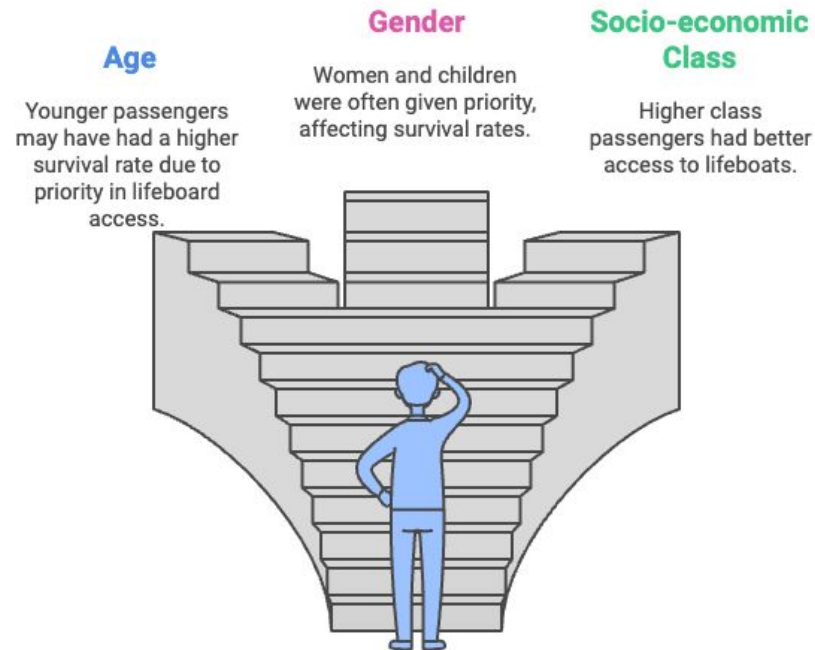
EDA of Tabular Data

Dataset to Explore: Titanic Dataset

The goal is to explore and clean the dataset so it can be used for predicting which passengers survived the titanic shipwreck.

How would you approach understanding this dataset?

What factors contributed to survival on the Titanic?





Load Dataset

```
import pandas as pd

# Load the Titanic dataset
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
titanic_df = pd.read_csv(url)

# Display the first few rows
titanic_df.head()
```

	PassengerId	Survived	Pclass		Name	Sex	Age	SibSp	Parch		Ticket	Fare	Cabin	Embarked
0	1	0	3		Braund, Mr. Owen Harris	male	22.0	1	0		A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0		1	0		PC 17599	71.2833	C85	C
2	3	1	3		Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN		S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0		1	0		113803	53.1000	C123	S
4	5	0	3		Allen, Mr. William Henry	male	35.0	0	0		373450	8.0500	NaN	S



Overview of Dataset Contents

Feature	Description	Possible Values
PassengerId	Unique identifier for each passenger	Integer (e.g., 1, 2, 3, ...)
Survived	Survival status (target variable)	0 = Did not survive, 1 = Survived
Pclass	Ticket class (proxy for socioeconomic status)	1 = First, 2 = Second, 3 = Third
Name	Passenger's full name	String (e.g., "Braund, Mr. Owen Harris")
Sex	Passenger's gender	Male, Female
Age	Passenger's age	Float (e.g., 22.0, 35.0), or NaN (missing)
SibSp	Number of siblings/spouses aboard	Integer (e.g., 0, 1, 2, ...)
Parch	Number of parents/children aboard	Integer (e.g., 0, 1, 2, ...)
Ticket	Ticket number	Alphanumeric (e.g., "A/5 21171")
Fare	Passenger's fare	Float (e.g., 7.25, 71.83)
Cabin	Cabin number	Alphanumeric (e.g., "C85"), or NaN (missing)
Embarked	Port of embarkation	C = Cherbourg, Q = Queenstown, S = Southampton



Initial Feature Pruning: What makes sense to stay?

Now that we took a look at our dataset, at first glance, are all features valuable to the prediction of the survival?

Do not contribute information, so we can remove them from the dataset.

Feature	Description	Possible Values
PassengerId	Unique identifier for each passenger	Integer (e.g., 1, 2, 3, ...)
Survived	Survival status (target variable)	0 = Did not survive, 1 = Survived
Pclass	Ticket class (proxy for socioeconomic status)	1 = First, 2 = Second, 3 = Third
Name	Passenger's full name	String (e.g., "Braund, Mr. Owen Harris")
Sex	Passenger's gender	Male, Female
Age	Passenger's age	Float (e.g., 22.0, 35.0), or NaN (missing)
SibSp	Number of siblings/spouses aboard	Integer (e.g., 0, 1, 2, ...)
Parch	Number of parents/children aboard	Integer (e.g., 0, 1, 2, ...)
Ticket	Ticket number	Alphanumeric (e.g., "A/5 21171")
Fare	Passenger's fare	Float (e.g., 7.25, 71.83)
Cabin	Cabin number	Alphanumeric (e.g., "C85"), or NaN (missing)
Embarked	Port of embarkation	C = Cherbourg, Q = Queenstown, S = Southampton



Dropping Useless Features

Note that we will keep the passenger ID until the end in case it is useful for indexing or grouping.

```
titanic_df = titanic_df.drop(['PassengerId', 'Name'], axis=1)
titanic_df.head()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	1	1	female	38.0	1	0	PC 17599	71.2833	C85	C
2	1	3	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	1	1	female	35.0	1	0	113803	53.1000	C123	S
4	0	3	male	35.0	0	0	373450	8.0500	NaN	S



Tabular Statistical Analysis: Distribution Analysis

In distribution analysis for tabular data, we need to explore:

1. Data size
 - a. Number of features
 - b. Number of data points
2. Feature statistics
 - a. Number of missing values
 - b. Summary statistics like mean, median, mode, variance, standard deviation, and percentiles
3. Feature Skewness (bias)
 - a. Degree of skewness (positive or negative)
 - b. Data Bias

Statistical Analysis: Distribution Analysis

Distribution Analysis

Feature Statistics

Feature statistics provide a summary of each feature's characteristics, helping identify trends, central tendencies, and potential issues like missing data or anomalies.

Data Size

Understanding the structure of the dataset is crucial to assess its completeness and suitability for analysis.



Feature Skewness

Feature skewness measures the asymmetry of the feature distribution. It indicates whether the data is biased toward lower or higher values, which can influence model performance.

Distribution Analysis: Data Size

Columns = Features / Rows = Data Points or Data Samples.

```
# Data size: Number of features and number of data points
num_features = titanic_df.shape[1]
num_data_points = titanic_df.shape[0]

print(f"Number of features: {num_features}")
print(f"Number of data points: {num_data_points}")
```

```
Number of features: 12
Number of data points: 891
```

Distribution Analysis: Feature Statistics

1. **Missing Values:** Missing data can significantly impact the performance of machine learning algorithms, leading to skewed predictions and compromised insights.

```
missing_values = titanic_df.isnull().sum()  
print("Number of missing values for each feature:\n", missing_values)
```

```
Number of missing values for each feature:
```

```
  PassengerId      0  
  Survived        0  
  Pclass         0  
  Name           0  
  Sex            0  
  Age           177  
  SibSp          0  
  Parch          0  
  Ticket         0  
  Fare           0  
  Cabin         687  
  Embarked        2  
dtype: int64
```

Handling Missing Values

There are many approaches to **filling missing values** (which would skew and confuse the model if left unhandled):

1. Based on **central tendency**: Fill missing values with the mean, median, or mode of the column.
2. **Conditional imputation**: Use other features to estimate the missing feature. For instance, calculate the median age for each Pclass and Sex combination and use that for imputation.
3. **Prediction**: Train a predictive model to predict the value of the missing rows in a feature.
4. **Drop**: If the feature has too many missing values, then we don't have enough information on the distribution of that feature, and we might as well drop it.

Handling Missing Values

1. **Age Missing Values:** From intuition, we know that age could be of high **influence** on the chance of survival. The lifeboats could have been prioritized for wealthy women and children first. So, we will choose conditional imputation to fill the missing ages.

Any ideas on what possibilities we can use for the conditions?

Handling Missing Values

1. Age missing values: We will use the median of each sex in each Pclass to fill the missing values.

```
age_median = titanic_df.groupby(['Pclass', 'Sex'])['Age'].median()
age_median
```

		Age
Pclass	Sex	
1	female	35.0
	male	40.0
2	female	28.0
	male	30.0
3	female	21.5
	male	25.0

```
titanic_df['Age'] = titanic_df.apply(
    lambda row: age_median[row['Pclass'], row['Sex']] if pd.isnull(row['Age']) else row['Age'], axis=1
)
missing_values = titanic_df.isnull().sum()
print("Number of missing values for each feature:\n", missing_values)
```

Number of missing values for each feature:

```
PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age             0
SibSp           0
Parch           0
Ticket          0
Fare            0
Cabin          687
Embarked        2
```

Handling Missing Values

2. **Cabin:** In the cabin number, the first letter of the cabin number indicates which wing the cabin was in. We can infer the cabin letter from the Pclass since more expensive tickets would have booked more expensive cabins.

How could we infer the cabin number?

Handling Missing Values

2. **Cabin:** The unique cabin number does not bring us any value in the prediction of the likelihood of survival, especially because it creates too many sparse values.

So, we will keep the first letter of the cabin ID and remove the unique numbers.

```
# Step 1: Extract deck information
titanic_df['Deck'] = titanic_df['Cabin'].str[0] # Extract first letter
# Remove any whitespace or invalid entries
titanic_df['Deck'] = titanic_df['Deck'].replace(' ', None)

# Step 2: Find the most common deck (mode) for each Pclass
deck_mode_by_pclass = titanic_df.groupby('Pclass')['Deck'].agg(lambda x: x.mode()[0] if not x.mode().empty else None)
print("Most common deck for each Pclass:\n", deck_mode_by_pclass)

# Step 3: Impute missing values in the Deck column based on Pclass
def impute_deck(row):
    if pd.isnull(row['Deck']):
        return deck_mode_by_pclass[row['Pclass']]
    return row['Deck']

titanic_df['Deck'] = titanic_df.apply(impute_deck, axis=1)
```

```
Most common deck for each Pclass:
Pclass
1      C
2      F
3      F
Name: Deck, dtype: object
```

Handling Missing Values

3. **Embarked:** The port of embarkation may also be influenced by Pclass as employees on the titanic may have onboarded using a different port than the VIP guests. So, we will replace the missing values using the same plan as cabin.

```
# Step 1: Find the most common embarkation point (mode) for each Pclass
embarked_mode_by_pclass = titanic_df.groupby('Pclass')['Embarked'].agg(lambda x: x.mode()[0] if not x.mode().empty else None)
print("Most common embarkation point for each Pclass:\n", embarked_mode_by_pclass)

# Step 2: Impute missing values in the Embarked column based on Pclass
def impute_embarked(row):
    if pd.isnull(row['Embarked']):
        return embarked_mode_by_pclass[row['Pclass']]
    return row['Embarked']

titanic_df['Embarked'] = titanic_df.apply(impute_embarked, axis=1)
```

```
Most common embarkation point for each Pclass:
Pclass
1      S
2      S
3      S
Name: Embarked, dtype: object
```

Handling Missing Values

Even though, intuitively, the port may have been influenced by the Pclass, the results showed that all classes used S port more frequently. Thus, we will use the most frequent (or average) port to fill the missing values.

This **hypothesis-testing-results cycle** is a crucial step in **EDA**. Not all decisions will be obvious, straightforward, or require 1 iteration.

Distribution Analysis: Feature Statistics

2. Summary Statistics 1: Overview of the statistical properties of the features.

How many data points is this feature populated?

What is the average value of this feature?

On average, how far is each score from the mean?

What are the 25th, 50th, and 75th percentiles?

What is the range of the feature?

```
summary_stats = titanic_df.describe()
print("Summary statistics:\n")
summary_stats
```

Summary statistics:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200



Numerical Representation of Features

In the previous slide, we notice that only the **continuous features** have summary statistics. This is because they are represented in a **numerical format**. To move forward with updating our dataset uniformly, we need **all features** to have a numerical format. We will use label encoding to encode categorical features.

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()
titanic_df['Sex'] = label_encoder.fit_transform(titanic_df['Sex'])
titanic_df['Embarked'] = label_encoder.fit_transform(titanic_df['Embarked'])
titanic_df['Deck'] = label_encoder.fit_transform(titanic_df['Deck'])
titanic_df.head()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked	Deck
0	0	3	1	22.0	1	0	A/5 21171	7.2500	2	5
1	1	1	0	38.0	1	0	PC 17599	71.2833	0	2
2	1	3	0	26.0	0	0	STON/O2. 3101282	7.9250	2	5
3	1	1	0	35.0	1	0	113803	53.1000	2	2
4	0	3	1	35.0	0	0	373450	8.0500	2	5

Numerical Representation of Features

There is still one feature we did not encode: Ticket.

While it may seem like an uninformative categorical variable at first glance, it can provide useful insights.

1. Some tickets have prefixes (e.g., "PC", "STON"). These prefixes may indicate different classes, routes, or embarkation points.
2. Type of ticket (family, group, individual): Group size and relationships could influence survival rates due to social dynamics during the evacuation.

Numerical Representation of Features

```
# Extract ticket prefixes
titanic_df['Ticket_Prefix'] = titanic_df['Ticket'].str.extract(r'([A-Za-z\.\./]+)', expand=False).fillna('None')

# Extract ticket group size
ticket_group_size = titanic_df.groupby('Ticket')['PassengerId'].transform('count')
titanic_df['Ticket_Group_Size'] = ticket_group_size

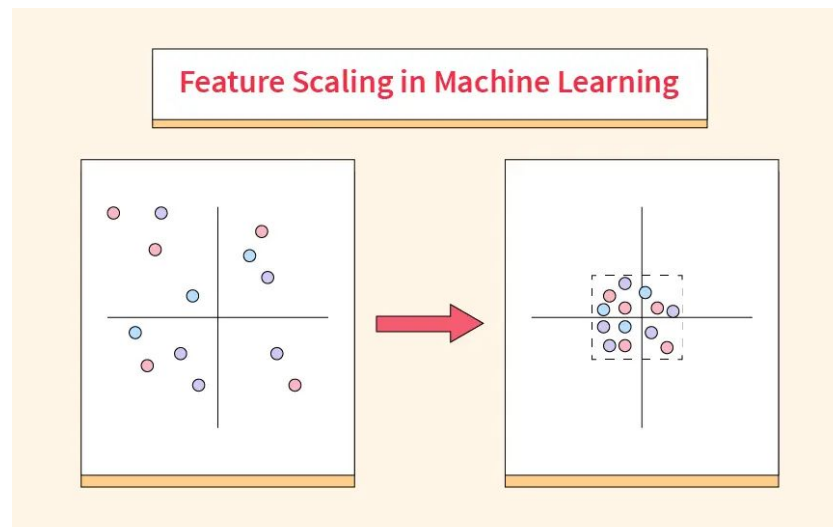
# Display sample of Ticket-related features
titanic_df[['Ticket', 'Ticket_Prefix', 'Ticket_Group_Size']].head()
```

	Ticket	Ticket_Prefix	Ticket_Group_Size
0	A/5 21171	A/	1
1	PC 17599	PC	1
2	STON/O2. 3101282	STON/O	1
3	113803	None	2
4	373450	None	1

Feature Range: Normalization

Since machine learning algorithms rely on **mathematical equations**, the statistical properties of each **input feature** significantly **influence the algorithm's training process** and the formulation of its predictions.

One of the most influential properties of a feature is its **scale or range**.



Feature Range: Normalization

Feeding high values for some features (for example price of a ticket) vs low values for some features (like number of siblings onboarded) will cause the model to **associate more importance to the higher value features**. Thus, we normalize the values of all features to be between 0 and 1.

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

```
from sklearn.preprocessing import MinMaxScaler
min_max_scaler = MinMaxScaler()

# Get the column names before transforming the data
original_columns = titanic_df.columns
titanic_df = min_max_scaler.fit_transform(titanic_df)

# Create DataFrame using the original column names
titanic_df = pd.DataFrame(titanic_df, columns=original_columns)
titanic_df.head()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Deck	Ticket_Prefix	Ticket_Group_Size
0	0.0	1.0	1.0	0.271174	0.125	0.0	0.014151	1.0	0.714286	0.076923	0.000000
1	1.0	0.0	0.0	0.472229	0.125	0.0	0.139136	0.0	0.285714	0.410256	0.000000
2	1.0	1.0	0.0	0.321438	0.000	0.0	0.015469	1.0	0.714286	0.871795	0.000000
3	1.0	0.0	0.0	0.434531	0.125	0.0	0.103644	1.0	0.285714	0.358974	0.166667
4	0.0	1.0	1.0	0.434531	0.000	0.0	0.015713	1.0	0.714286	0.358974	0.000000

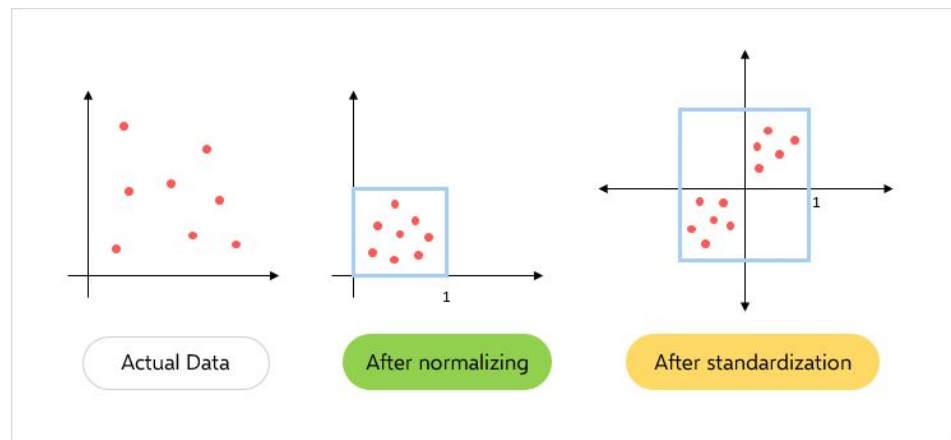
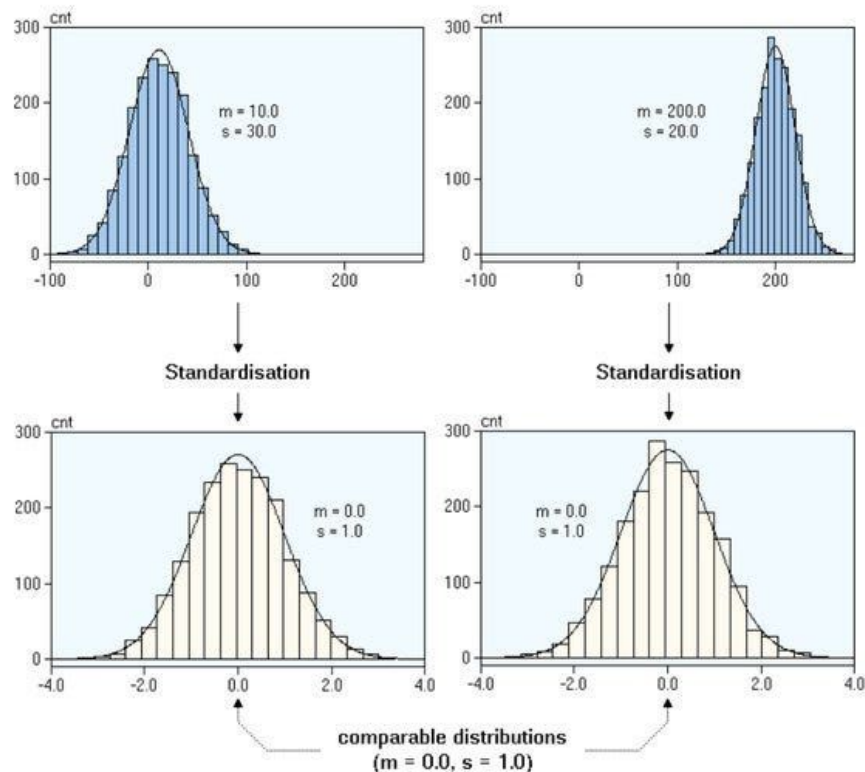
Distribution Analysis: Feature Statistics

3. **Mode:** The mode is the most common number that appears in your set of data. Checking the mode allows you to predict or understand any bias that might happen.

```
mode_values = titanic_df.mode().iloc[0]  
print("Mode values for each feature:\n", mode_values)
```

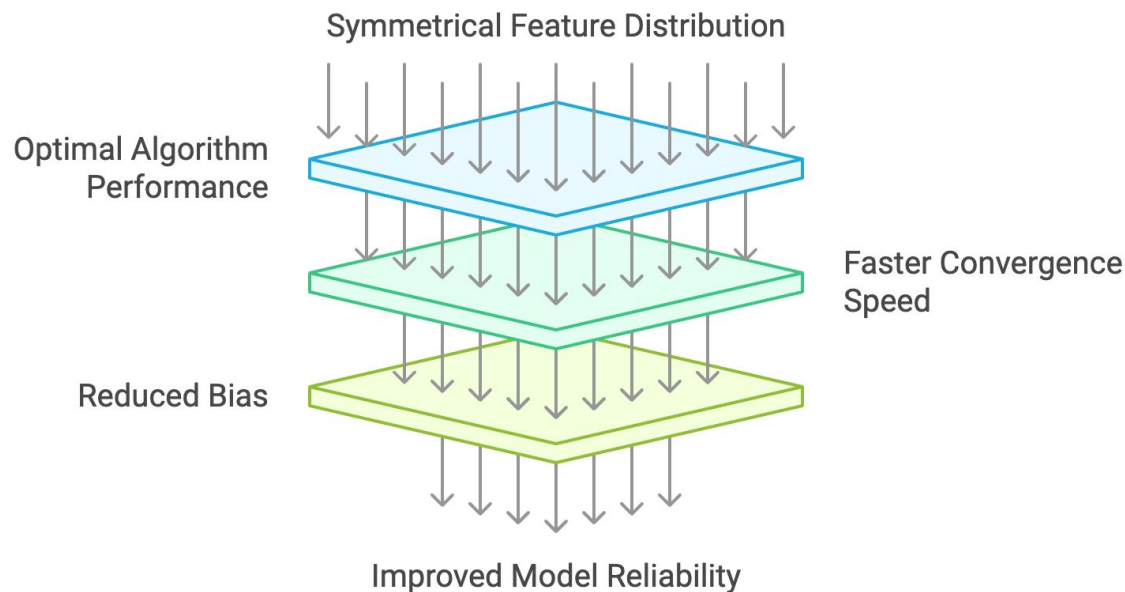
```
Mode values for each feature:  
Survived      0.000000  
Pclass        1.000000  
Sex            1.000000  
Age           0.308872  
SibSp         0.000000  
Parch         0.000000  
Fare          0.015713  
Embarked      1.000000  
Deck          0.714286  
Ticket_Prefix 0.358974  
Ticket_Group_Size 0.000000  
Name: 0, dtype: float64
```

Distribution Analysis: Feature Skewness



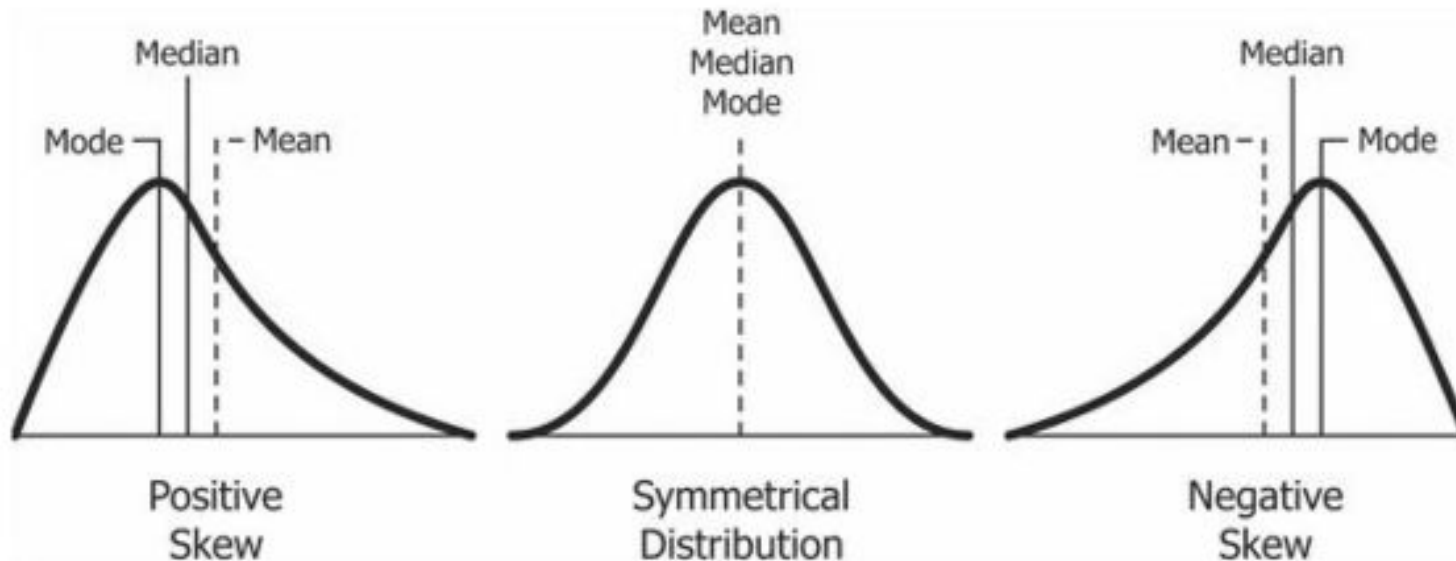
Distribution Analysis: Feature Skewness

Benefits of Symmetrical Distributions in Machine Learning



Distribution Analysis: Feature Skewness

There are two types of skews: **positive** and **negative**.





Distribution Analysis: Feature Skewness

Some skewness in features is normal (like positive skew for binary categorical features). Nonetheless, we need all features to follow a symmetrical distribution.

```
import numpy as np

# Apply log transformation (log1p handles zero values safely)
# For highly skewed features like SibSp, Parch, and Fare, apply log1p transformation

# Apply log1p transformation to handle zero values in the data
titanic_df['SibSp_log'] = np.log1p(titanic_df['SibSp'])
titanic_df['Parch_log'] = np.log1p(titanic_df['Parch'])
titanic_df['Fare_log'] = np.log1p(titanic_df['Fare'])
titanic_df['Ticket_Group_Size'] = np.log1p(titanic_df['Ticket_Group_Size'])

# Recalculate skewness after transformation
skewness_after_log = titanic_df[['SibSp_log', 'Parch_log', 'Fare_log', 'Ticket_Group_Size']].skew()

# Print the skewness after transformation
print("Skewness after log transformation:")
print(skewness_after_log)
```



```
Skewness after log transformation:
SibSp_log      1.408772
Parch_log      1.492582
Fare_log       1.485546
Ticket_Group_Size 1.013619
dtype: float64
```

Statistical Analysis: Outlier Detection

Outliers can introduce **noise** into the data, making it **difficult** for a machine learning model to learn the true underlying patterns. This can lead to **reduced accuracy and performance**.

There are many ways to detect outliers in the feature. We will take a look at two **statistical** methods to detect outliers:

1. Z-score Test
2. Interquartile Range (IQR) Method



Outlier Detection: Z-Score Test

1- Calculate the z-score of each data point

$$z = \frac{x - \mu}{\sigma}$$

μ = Mean

σ = Standard Deviation

2- Data points with $|Z| > \text{threshold}$ (rule of thumb: threshold = 3) are considered outliers.

Why threshold=3?

In a normal distribution:

- ~68% of the data lies within 1 standard deviation ($|Z| \leq 1$).
- ~95% of the data lies within 2 standard deviations ($|Z| \leq 2$).
- ~99.7% of the data lies within 3 standard deviations ($|Z| \leq 3$).



Outlier Detection: Z-Score Test

We will use Fare feature as an example:

```
# Z-score method
from scipy.stats import zscore
titanic_df['Fare_zscore'] = zscore(titanic_df['Fare'])
outliers_zscore = titanic_df[np.abs(titanic_df['Fare_zscore']) > 3]
print("Outliers detected using Z-score:\n", outliers_zscore[['Fare', 'Fare_zscore']])
```

Outliers detected using Z-score:

	Fare	Fare_zscore
27	0.513342	4.647001
88	0.513342	4.647001
118	0.483128	4.335332
258	1.000000	9.667167
299	0.483128	4.335332
311	0.512122	4.634417
341	0.513342	4.647001
377	0.412821	3.610065
380	0.444099	3.932723
438	0.513342	4.647001
527	0.432884	3.817033
557	0.444099	3.932723
679	1.000000	9.667167
689	0.412503	3.606793
700	0.444099	3.932723
716	0.444099	3.932723
730	0.412503	3.606793
737	1.000000	9.667167
742	0.512122	4.634417
779	0.412503	3.606793

Outlier Detection: Interquartile Range (IQR) Method

In the IQR method, we consider everything outside the range of

$$[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$$

as an outlier. Where Q1 and Q3 are the 25th and 75th percentiles we saw in the summary statistics. And the $IQR = Q3 - Q1$.

```
# IQR method
Q1 = titanic_df['Fare'].quantile(0.25)
Q3 = titanic_df['Fare'].quantile(0.75)
IQR = Q3 - Q1
outliers_iqr = titanic_df[(titanic_df['Fare'] < Q1 - 1.5 * IQR) | (titanic_df['Fare'] > Q3 + 1.5 * IQR)]
print("Outliers detected using IQR:\n", outliers_iqr[['Fare']])
```

Outliers detected using IQR:

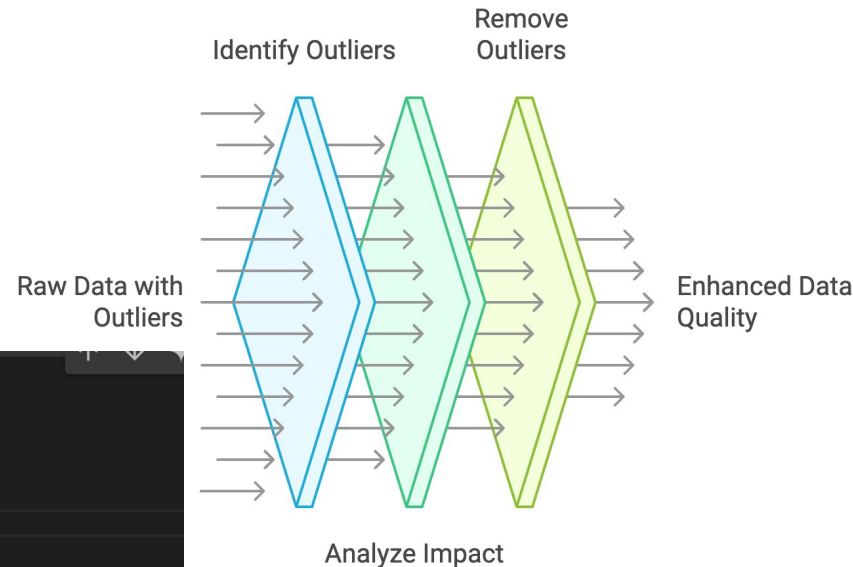
	Fare
1	0.139136
27	0.513342
31	0.285990
34	0.160387
52	0.149765
..	...
846	0.135753
849	0.173920
856	0.321798
863	0.135753
879	0.162314

Removing Outliers

If the outliers in a certain feature are undesirable (improbable data points that will confuse the model, then we need to remove them)

```
features_with_outliers = ['Age', 'Fare']  
  
for feature in features_with_outliers:  
    Q1 = titanic[feature].quantile(0.25)  
    Q3 = titanic[feature].quantile(0.75)  
    IQR = Q3 - Q1  
    lower_bound = Q1 - 1.5 * IQR  
    upper_bound = Q3 + 1.5 * IQR  
  
# Step 3: Filter out the outliers  
titanic = titanic[(titanic[feature] >= lower_bound) & (titanic[feature] <= upper_bound)]
```

Outlier Removal Process



Visualization Analysis

Another technique for understanding your data is through charts. You can visualize:

1. Distribution of features (categorical and continuous)
2. Outliers
3. Feature Interactions
4. Relationships between features and target output (if exists)

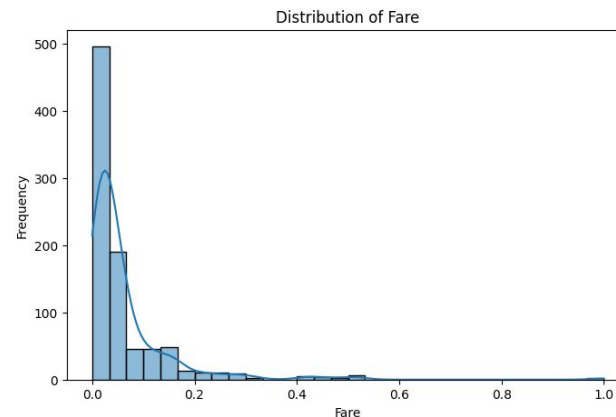
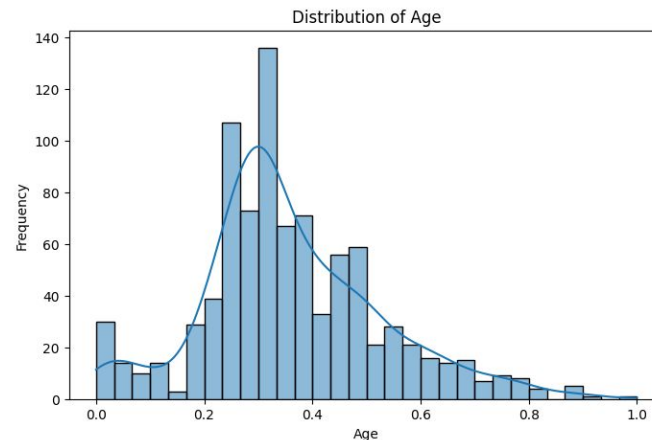
Visualization Analysis: Distribution of features

Let's plot some continuous features at first:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Select continuous features
continuous_features = ['Age', 'Fare']

# Plot distributions
for feature in continuous_features:
    plt.figure(figsize=(8, 5))
    sns.histplot(titanic_df[feature].dropna(), kde=True, bins=30)
    plt.title(f'Distribution of {feature}')
    plt.xlabel(feature)
    plt.ylabel('Frequency')
    plt.show()
```



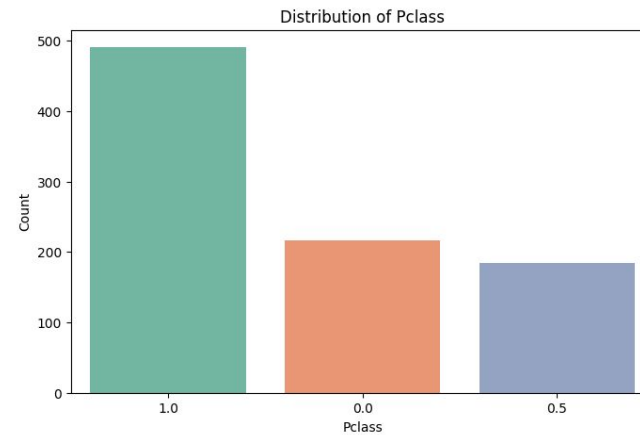
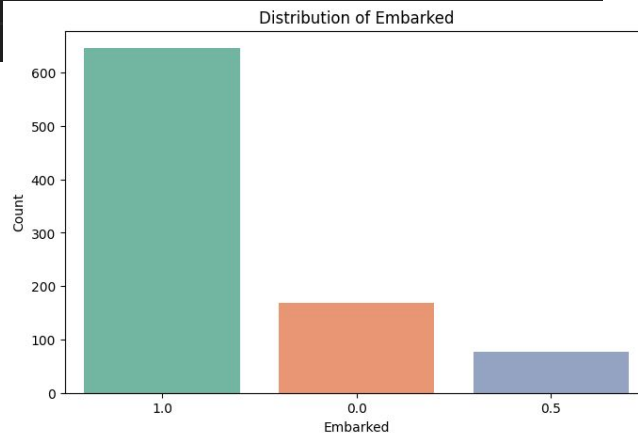
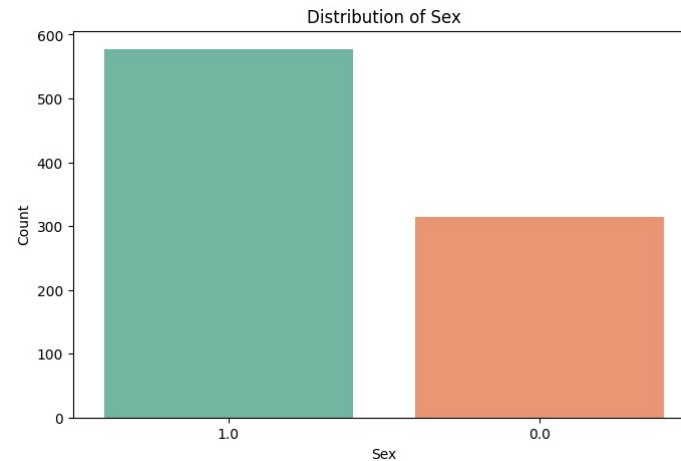


Visualization Analysis: Distribution of features

Distribution of categorical variables:

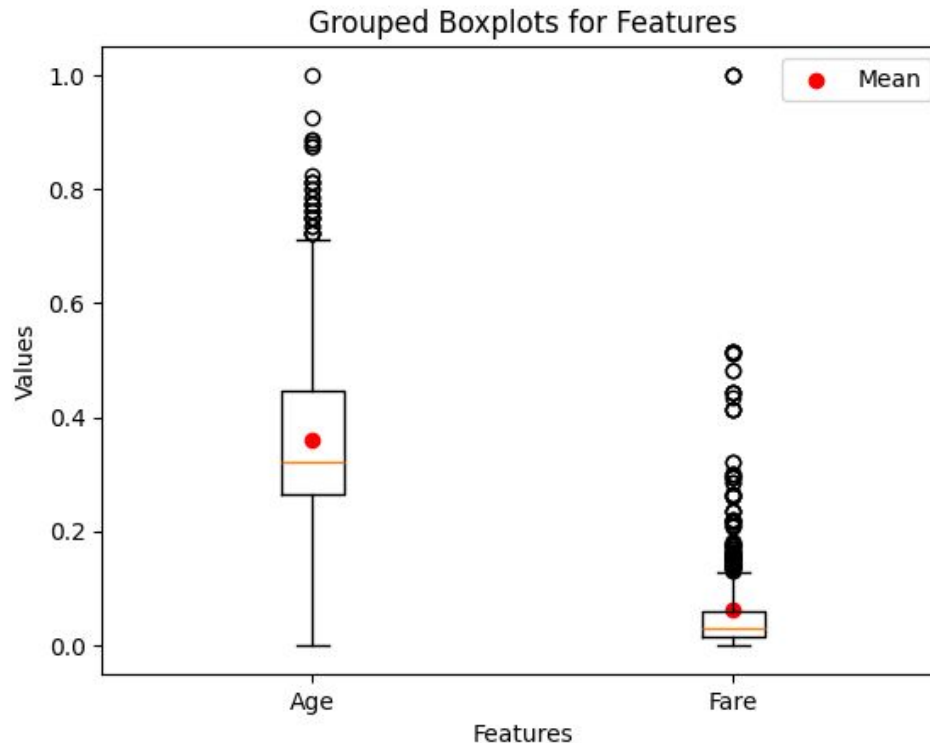
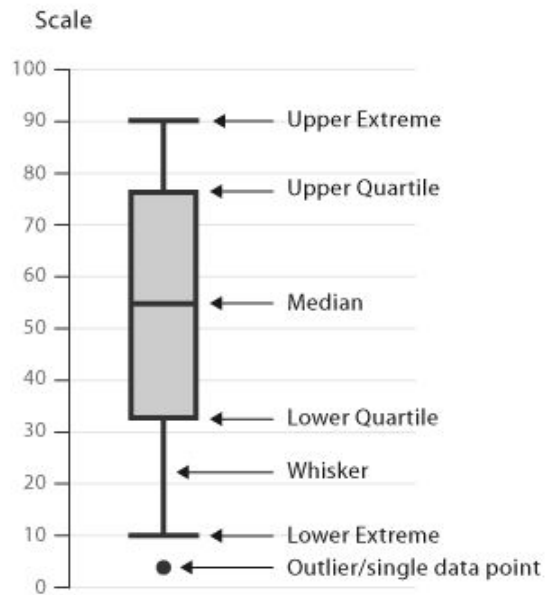
```
# Select categorical features
categorical_features = ['Sex', 'Pclass', 'Embarked']

# Plot distributions
for feature in categorical_features:
    plt.figure(figsize=(8, 5))
    sns.countplot(x=titanic_df[feature], palette="Set2",
                  legend=False, order=titanic_df[feature].value_counts().index)
    plt.title(f'Distribution of {feature}')
    plt.xlabel(feature)
    plt.ylabel('Count')
    plt.show()
```



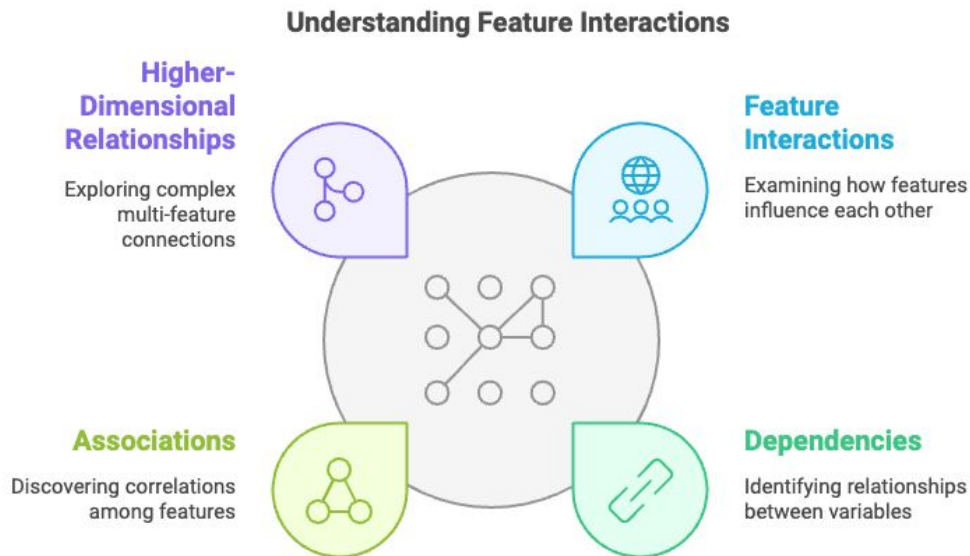
Visualization Analysis: Outliers

Boxplots are an easy way to check for outliers in a feature.



Visualization Analysis: Feature Interactions

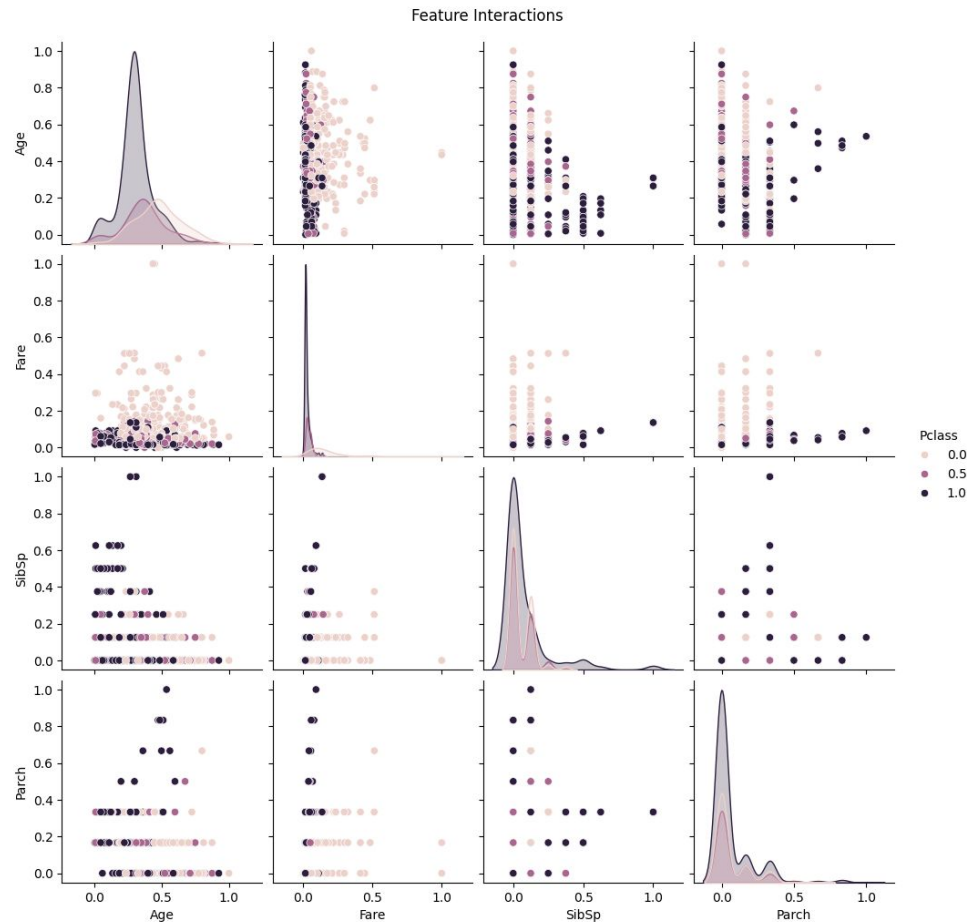
Examining interactions between features to uncover dependencies, associations, and higher-dimensional relationships.



Visualization Analysis: Feature Interactions

Interactions between features can be visualized using pair plots.

```
sns.pairplot(titanic_df[['Age', 'Fare', 'Pclass', 'SibSp', 'Parch']].dropna(),
              diag_kind='kde', hue='Pclass')
plt.suptitle('Feature Interactions', y=1.02)
plt.show()
```



Feature Interactions: Correlation of Features

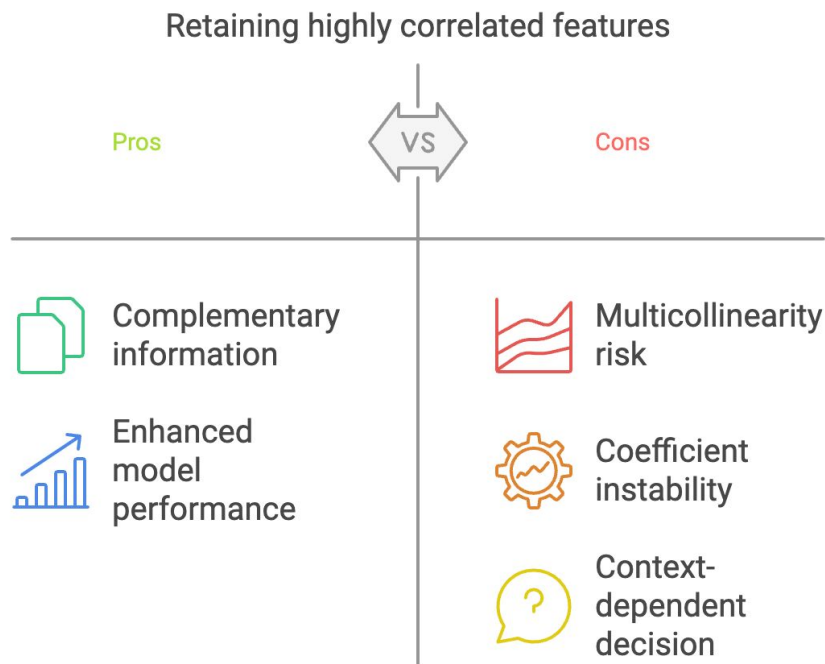
Feature correlation is a statistical concept that quantifies the **linear relationship** between **two or more variables**. It can range from -1 to 1, where -1 means a perfect **negative correlation**, 0 means **no correlation**, and 1 means a perfect **positive correlation**.

The linear relationship between features: $f1 = m.f2 + c$.

- If there is a negative correlation $m < 0$
- If there is no correlation $m = 0$
- If there is a positive correlation $m > 0$

Feature Interactions: Correlation of Features

If f_1 and f_2 have a high correlation (positive or negative), it is advisable to **remove** one of them or **combine** them to create a new feature f_3 that contains both information.



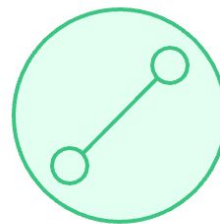
Feature Interactions: Correlation of Features

Having no linear correlation does not mean that the relationships don't have a connection. There can exist **non-linear relationships** within the data. However, we don't check for these relationships in most cases since they add value to the information instead of causing redundancy.



Non-Linear Relationships

Capture complex interactions



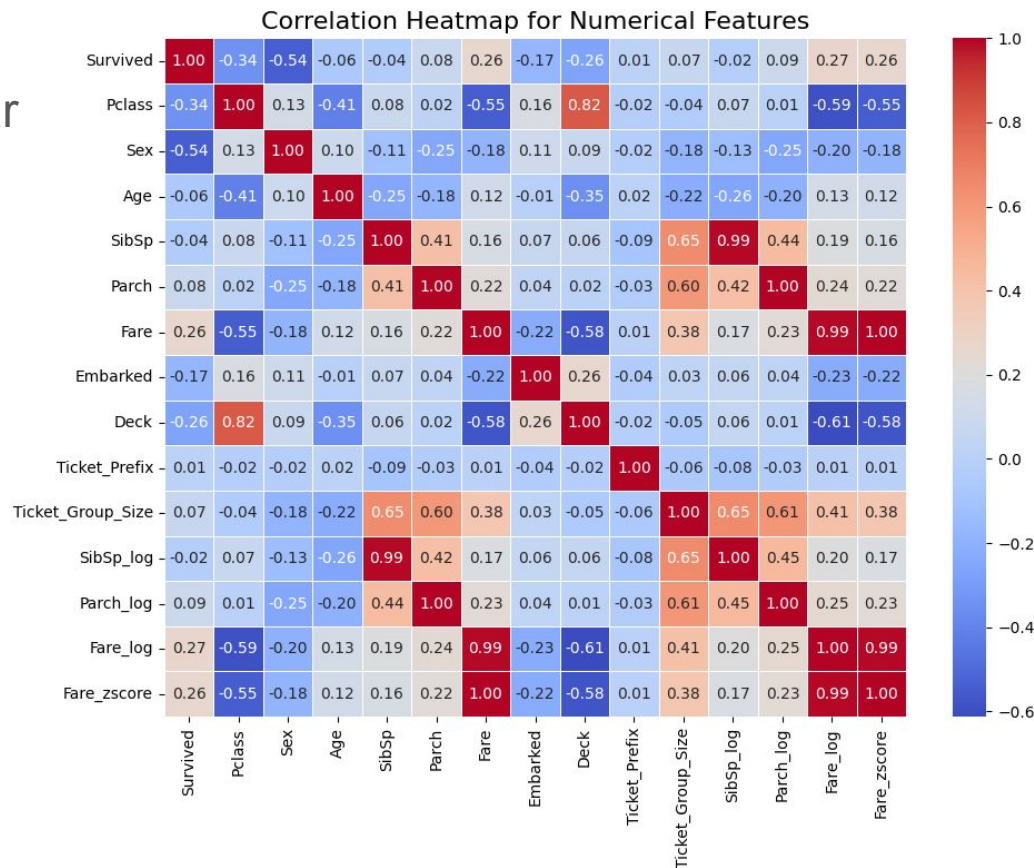
Linear Relationships

Simplify to straight lines

Feature Interactions: Correlation of Features

Heat maps show the level of linear correlation between two features.

It is unadvisable to have a high correlation between two features since we would be feeding the model repetitive information.



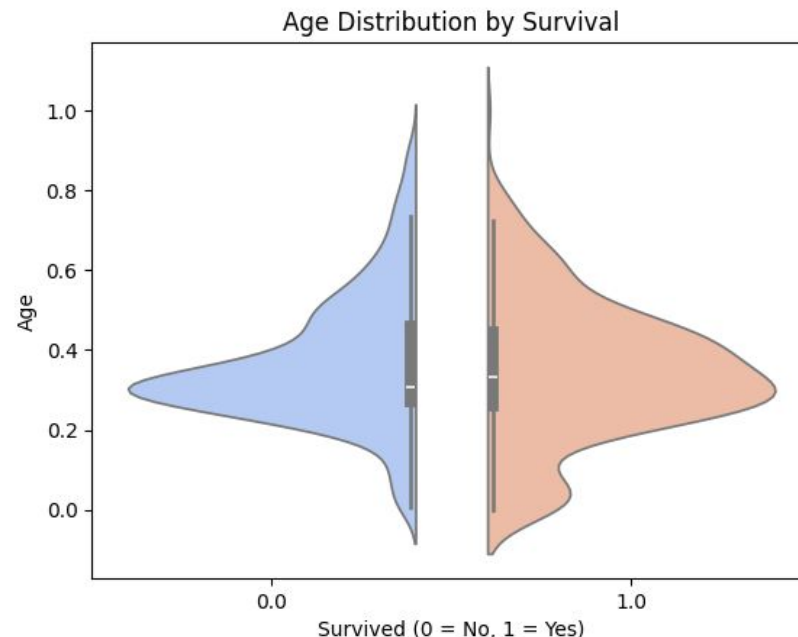
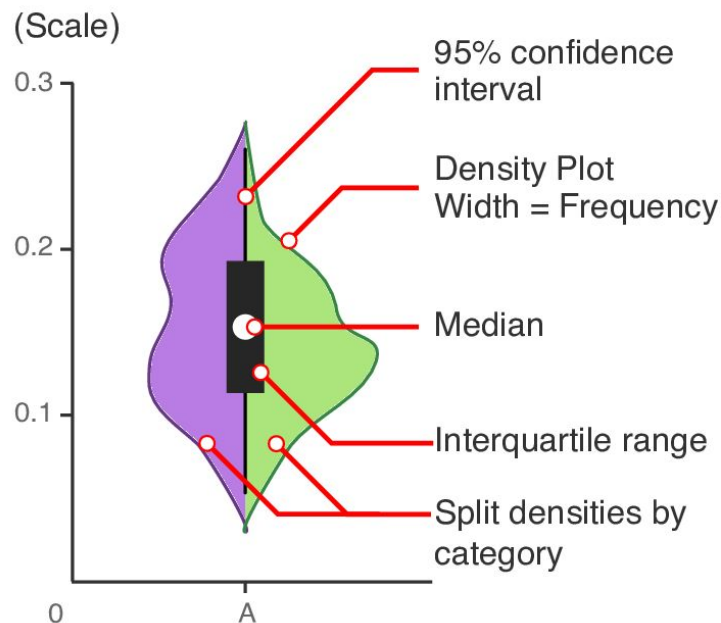
Visualization Analysis: Effect of Features on Output



Feature Type	Visualization	Why Use This?
Numerical vs Target	Violin Plot, Pairplot, Heatmap	Shows detailed distribution, relationships, and correlations.
Categorical vs Target	Count Plot, Bar Plot	Displays counts or proportions of categories by target group.
Mixed Features	Pairplot, Violin Plot	Shows interactions between features, especially with mixed data types.
Individual Data Points	Strip Plot	Visualizes individual observations, especially for smaller datasets.

Visualization Analysis: Effect of Features on Output

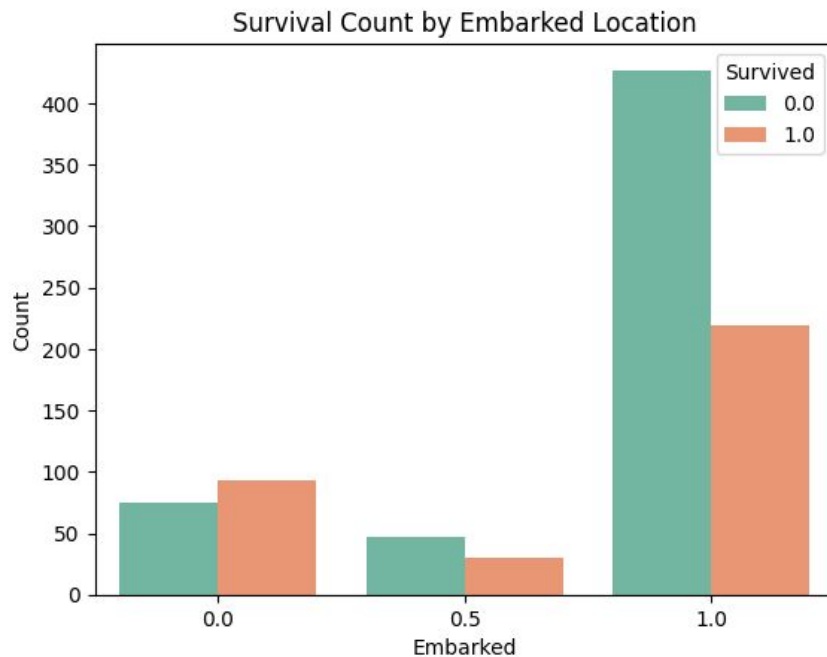
To visualize the effect of a continuous variable on the output, we will use a violin plot.



Visualization Analysis: Effect of Features on Output



For categorical features, we can use a count plot, which is the same as a bar plot, but fixing the y-axis as count or frequency.



Dimensionality of Features

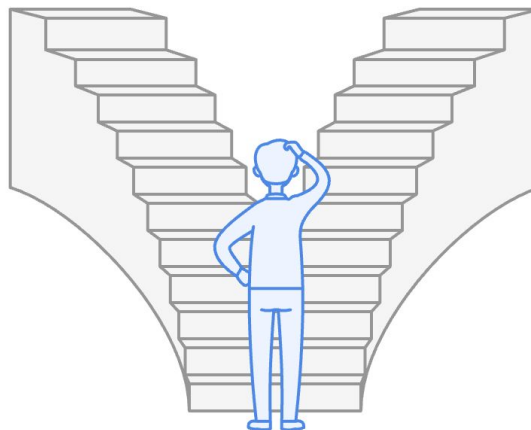
Dimensionality refers to the number of features or variables in a dataset.

Dimensionality Reduction

Reduces risk of overfitting and computational complexity but may lose important information.

Dimensionality Expansion

Adds complexity and captures more relationships but risks overfitting and increases computational demands.



Dimensionality of Features: Reduction

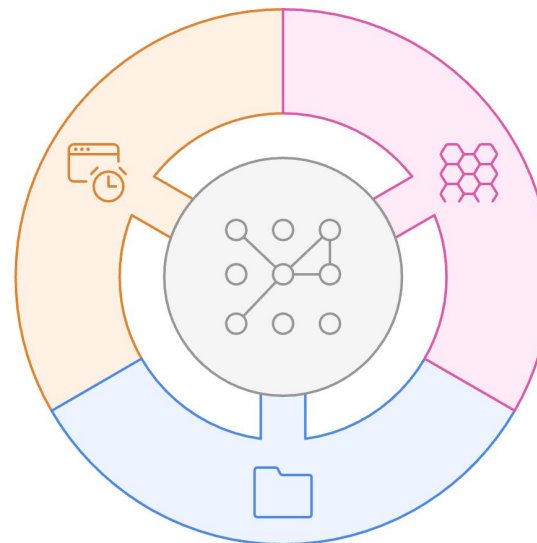
If the number of features is overly large in comparison to the number of data points, we need to consider reducing the dimensionality of our dataset.

Computational Complexity

More features result in longer training times and higher costs.

Overfitting

The model becomes overly complex, capturing noise instead of signal.



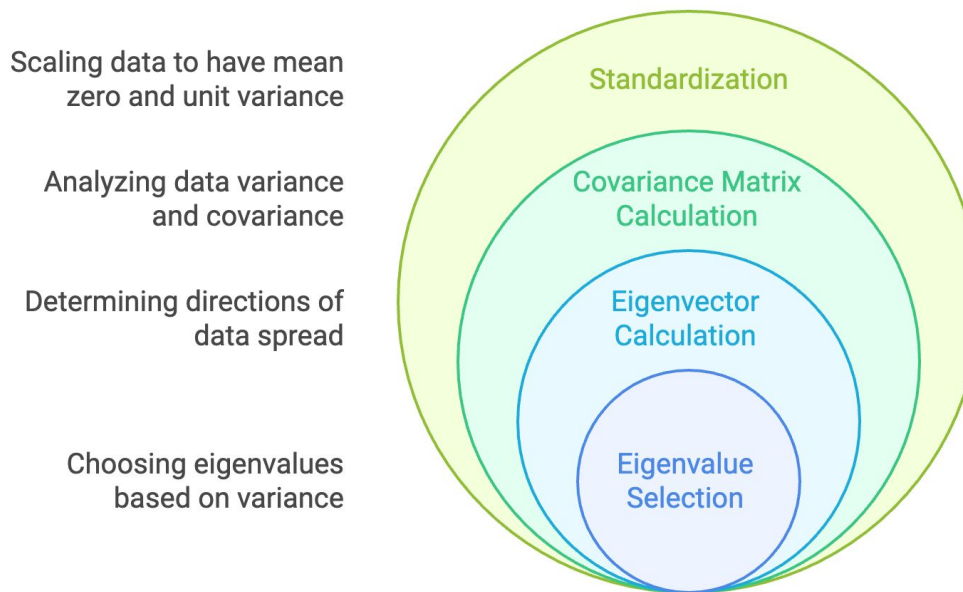
Curse of Dimensionality

Increased features lead to sparse data, complicating modeling efforts.

Dimensionality of Features: Reduction

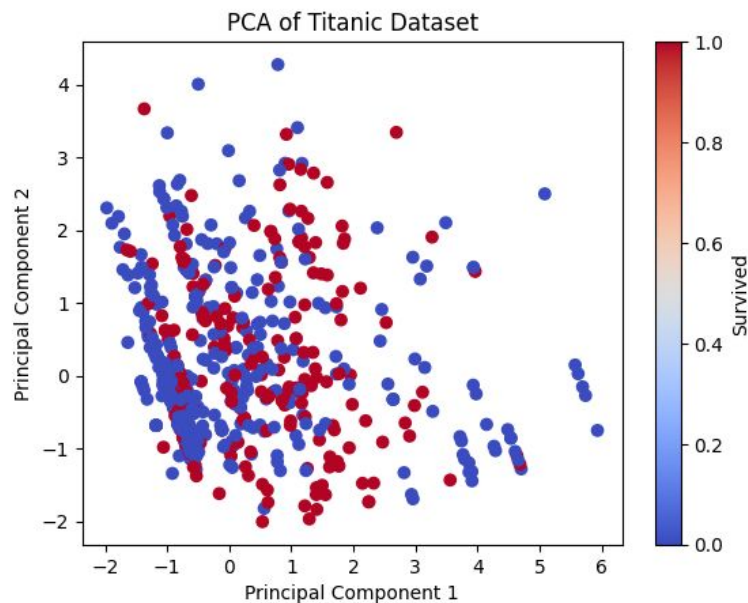
Principal Component Analysis (PCA) is a dimensionality reduction technique that computes the ideal number of features based on the level of variance we wish to retain in the data.

PCA Process Overview



Dimensionality of Features: Reduction

We will use PCA to reduce the 'Age', 'Fare', 'SibSp', 'Parch' in the titanic dataset to 2 features instead of 4.



```
# Step 1: Select numerical features and scale them
numerical_features = ['Age', 'Fare', 'SibSp', 'Parch']
titanic_scaled = StandardScaler().fit_transform(titanic[numerical_features].dropna())

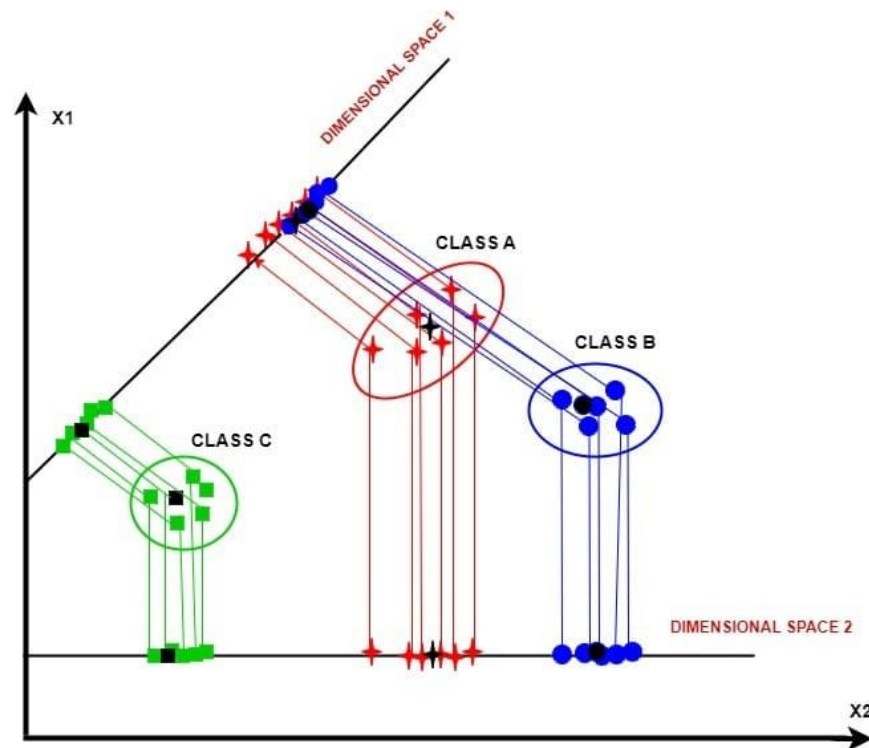
# Step 2: Apply PCA to reduce dimensions
pca = PCA(n_components=2) # Reduce to 2 components
titanic_pca = pca.fit_transform(titanic_scaled)

# Step 3: Check the explained variance
print(f"Explained variance ratio by each component: {pca.explained_variance_ratio_}")
print(f"Total variance explained: {sum(pca.explained_variance_ratio_)}")
```

```
Explained variance ratio by each component: [0.45650756 0.28297632]
Total variance explained: 0.7394838827303434
```

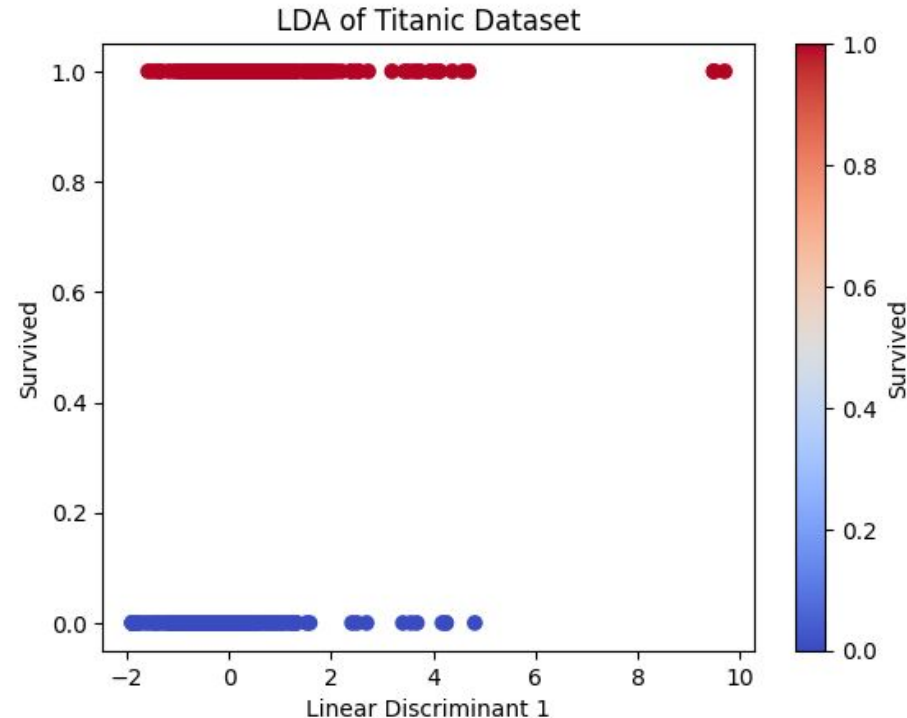
Dimensionality of Features: Reduction

If the dataset was labeled (supervised learning), and the target output is discrete (classification) then we can use **Linear Discriminant Analysis (LDA)** for dimensionality reduction which works by creating a **collection of linear separators (for C number of classes, we create $C-1$ separators)** for the target class using a combination of the features. These separators are the new features.



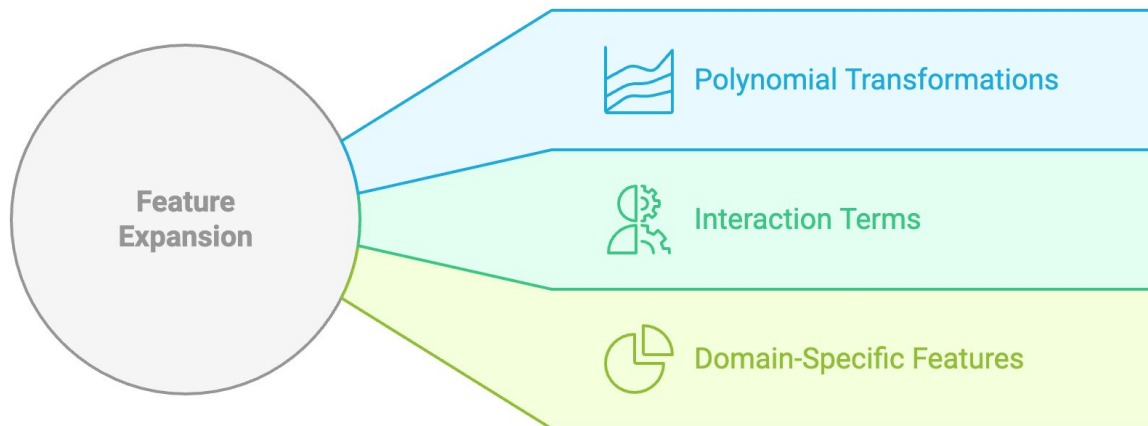
Dimensionality of Features: Reduction

Applying LDA on the titanic dataset to get 1 Linear Discriminant.



Dimensionality of Features: Expansion

Feature expansion refers to the process of generating new features from the existing dataset to capture more information and relationships within the data. In some cases, increasing the complexity and dimensionality of the data could allow the model to understand the patterns more clearly.



Dimensionality of Features: Expansion

Polynomial Features: Generate a new feature matrix consisting of all polynomial combinations of the features with degree less than or equal to the specified degree.

For example, if an input sample is two dimensional and of the form $[a, b]$, the degree-2 polynomial features are $[1, a, b, a^2, ab, b^2]$.

```
from sklearn.preprocessing import PolynomialFeatures

# Step 1: Create Polynomial Features
poly = PolynomialFeatures(degree=2, interaction_only=False)
titanic_poly = poly.fit_transform(titanic[numerical_features].dropna())

# Step 2: Check the new feature set
poly_feature_names = poly.get_feature_names_out(numerical_features)
print(f"New feature names: {poly_feature_names}")
```

```
New feature names: ['1' 'Age' 'Fare' 'SibSp' 'Parch' 'Age^2' 'Age Fare' 'Age SibSp'
'Age Parch' 'Fare^2' 'Fare SibSp' 'Fare Parch' 'SibSp^2' 'SibSp Parch'
'Parch^2']
```


Dimensionality of Features: Expansion

Techniques like one-hot encoding or target encoding can be used to convert categorical variables into numerical formats, allowing models to utilize these features effectively.

For Embarked, suppose the original values are ['C', 'S', 'Q']. After encoding:

- Embarked_C and Embarked_S are created.
- If both are 0, the observation belongs to the dropped category (Embarked_Q)

EDA For Text Data

Text Dataset: SPAM/HAM

The Spam/Ham Dataset is a classification dataset where we use an email's content to detect if it is Spam (Junk) or Ham (Not Spam).

```
# Importing necessary libraries
import pandas as pd

# Load the dataset
url = "https://raw.githubusercontent.com/mohitgupta-101/Kaggle-SMS-Spam-Collection-Dataset-/master/spam.csv"
data = pd.read_csv(url, encoding="ISO-8859-1")

# Data preview and cleaning
data = data.rename(columns={"v1": "label", "v2": "message"})
data = data[["label", "message"]] # Keeping only relevant columns
data.head()
```

	label	message	
0	ham	Go until jurong point, crazy.. Available only ...	
1	ham	Ok lar... Joking wif u oni...	
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	
3	ham	U dun say so early hor... U c already then say...	
4	ham	Nah I don't think he goes to usf, he lives aro...	

Statistical Analysis for Text Data

Since text data is composed of strings for each row, what are the statistical properties of text that we can explore?

Some text statistics that we can explore in our corpus:

- Word frequency
- Sentence length
- Average word length
- Average number of words

Statistical Analysis: Summary Statistics

To help understand the distribution properties and distribution of the dataset, we will add character count, word count, and average word length as features in our dataset.

```
# Adding character and word count columns
data["char_count"] = data["message"].apply(len)
data["word_count"] = data["message"].apply(lambda x: len(x.split()))
data["avg_word_length"] = data["char_count"] / data["word_count"]
data.head()
```

	label	message	char_count	word_count	avg_word_length
0	ham	Go until jurong point, crazy.. Available only ...	111	20	5.550000
1	ham	Ok lar... Joking wif u oni...	29	6	4.833333
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	155	28	5.535714
3	ham	U dun say so early hor... U c already then say...	49	11	4.454545
4	ham	Nah I don't think he goes to usf, he lives aro...	61	13	4.692308

Statistical Analysis: Summary Statistics

Now that we have some statistics, we can take a look at the differences between out two classes:

```
# Summary statistics
char_word_stats = data[["label", "char_count", "word_count", "avg_word_length"]].groupby("label").mean()
print(char_word_stats)
```

label	char_count	word_count	avg_word_length
ham	71.023627	14.200622	5.070165
spam	138.866131	23.851406	5.946532

Statistical Analysis: Word Frequency

It would be useful to take a look at the most frequent words that occur in each class since the model would be frequently exposed to these words.

What do you think the most frequent words would contain?

Do all words hold the same contribution to the classification?



Statistical Analysis: Word Frequency

Stopwords are common words (e.g., "the," "is," "in," "and") that occur frequently in a language but usually **add little semantic meaning** to text analysis or natural language processing (NLP) tasks.

In our case, **classification**, we can **remove stop words** for a better understanding of the message in each text. In addition, we will also use a **word tokenizer** to chunk the text into individual words before we count their frequency.

```
# Function to clean and tokenize text
def clean_tokenize_text(text):
    stop_words = set(stopwords.words('english'))
    text = text.lower()
    text = ''.join([char for char in text if char not in string.punctuation])
    words = word_tokenize(text)
    return [word for word in words if word not in stop_words]

# Apply cleaning and tokenization
data["cleaned_tokens"] = data["message"].apply(clean_tokenize_text)
```


Stop Words: when to remove/keep

Should stopwords be removed
during text pre-processing?

Remove Stopwords

Reduces noise and computational complexity in tasks like text classification or topic modeling.



Retain Stopwords

Preserves context and nuance in tasks like sentiment analysis or translation.



Statistical Analysis: Word Frequency

In the tables, we have the most frequent 15 words for each class.

What can you infer from this step?

Top 15 Most Common Words in Ham Messages:

	Word	Frequency	
0	u	972	
1	im	449	
2	2	305	
3	get	303	
4	ltgt	276	
5	ok	272	
6	dont	257	
7	go	247	
8	got	243	
9	ur	240	
10	ill	236	
11	know	232	
12	like	229	
13	call	229	
14	come	224	

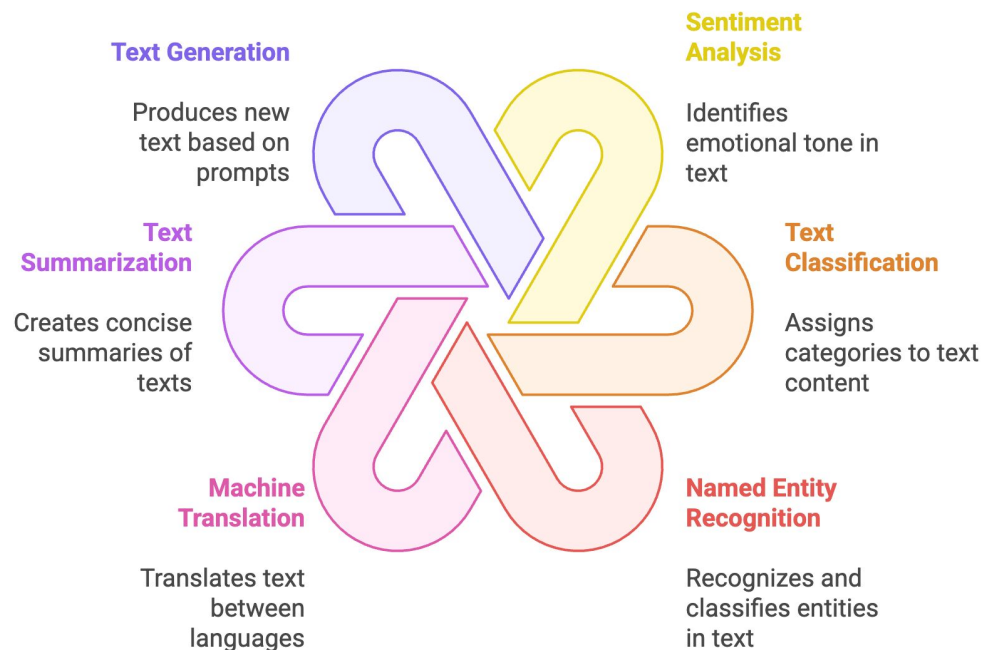
Top 15 Most Common Words in Spam Messages:

	Word	Frequency	
0	call	347	
1	free	216	
2	2	173	
3	txt	150	
4	u	147	
5	ur	144	
6	mobile	123	
7	text	120	
8	4	119	
9	claim	113	
10	stop	113	
11	reply	101	
12	prize	92	
13	get	83	
14	new	69	

Types of NLP Tasks

The steps to take for statistical analysis of text data heavily depends on the task.

Overview of Text Processing Applications



Visualization Analysis: Word Clouds

Word clouds are visualizations that display the **most frequently occurring words** in a text dataset, with their size reflecting their frequency or importance.

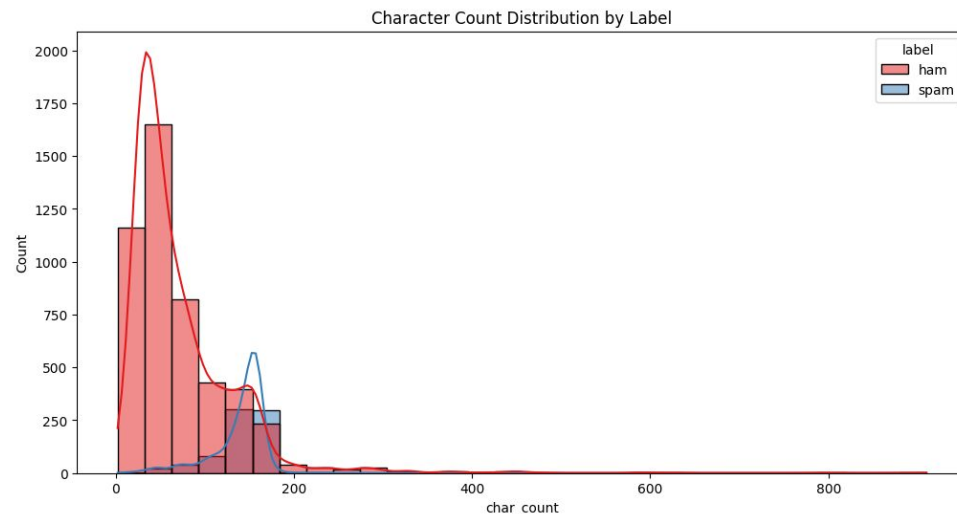
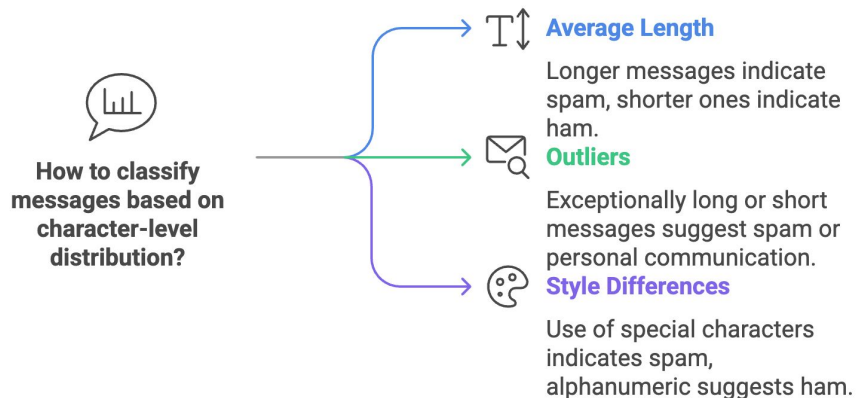
Word Cloud for Spam Messages



Word Cloud for Ham Messages

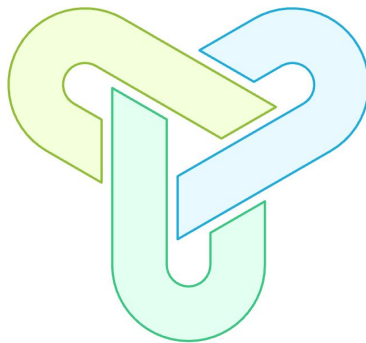


Visualization Analysis: Character Distributions



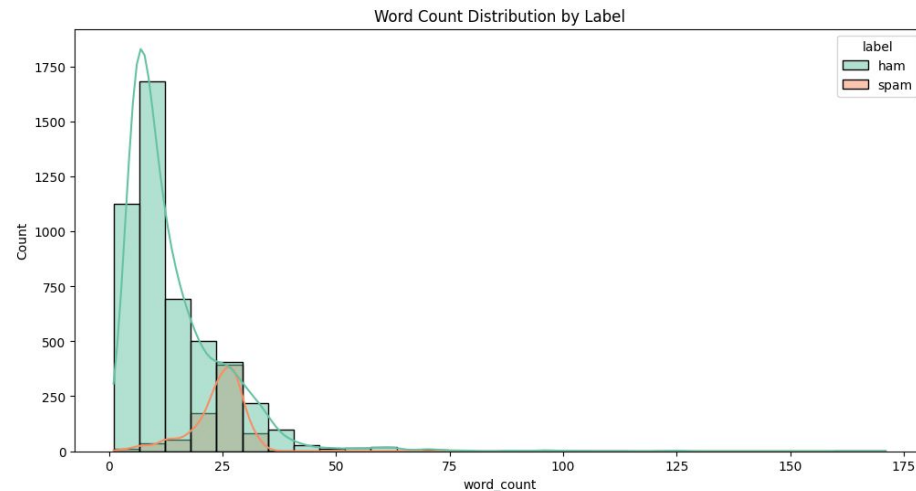
Visualization Analysis: Word Distributions

Message Composition
Highlights spam's urgency and ham's conversational elements



Vocabulary Size
Reflects promotional terms in spam and broader vocabulary in ham

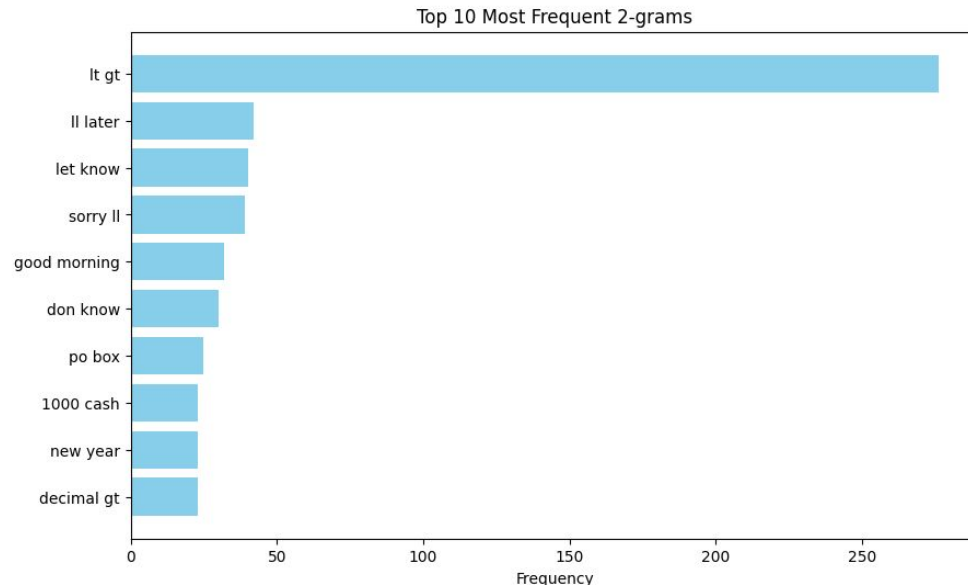
Density of Unique Words
Indicates repetition in spam and variability in ham



Visualization Analysis: N-grams

Ngrams are simply contiguous sequences of n words. If the number of words is two, it is called bigram. For 3 words it is called a trigram and so on.

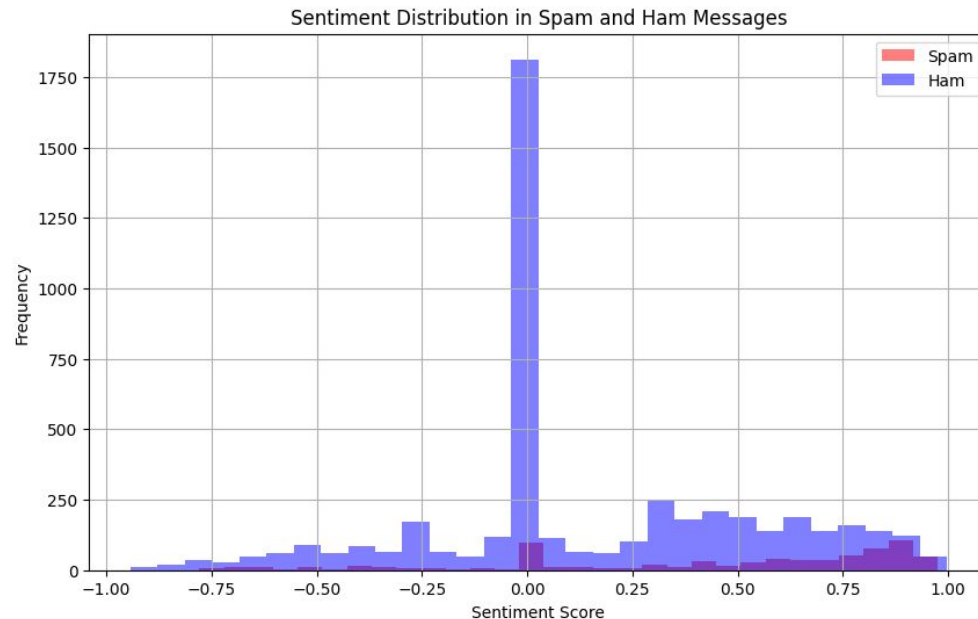
Looking at most frequent n-grams can give you a better understanding of the **context** in which the word was used.



Visualization Analysis: Sentiment Distribution

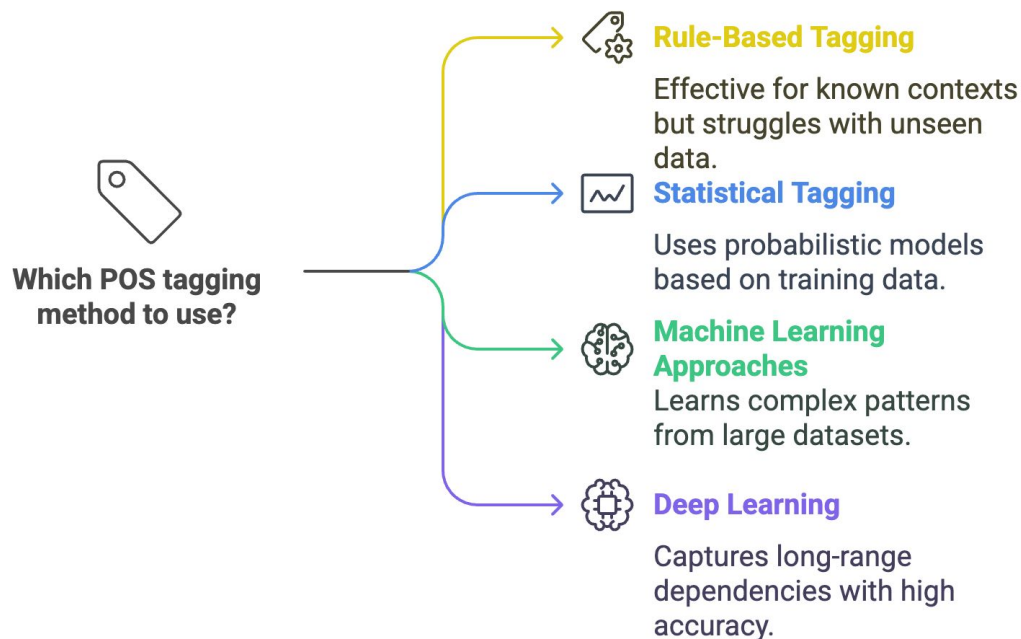
Let's analyze the sentiment (**positive**, **negative**, or **neutral**) of each message and visualize how sentiment varies across spam and ham messages.

- -1.0 represents the most negative sentiment
- 0.0 represents neutral sentiment
- +1.0 represents the most positive sentiment



Part of Speech (PoS)

Part-of-speech tagging (POS tagging or PoS tagging or POST), also called grammatical tagging is the process of **marking up a word in a text (corpus)** as corresponding to a **particular part of speech**, based on both its definition and its context.





POS Tagging

Tag	Description	Example	Tag	Description	Example	Tag	Description	Example
CC	coordinating conjunction	<i>and, but, or</i>	PDT	predeterminer	<i>all, both</i>	VBP	verb non-3sg present	<i>eat</i>
CD	cardinal number	<i>one, two</i>	POS	possessive ending	<i>'s</i>	VBZ	verb 3sg pres	<i>eats</i>
DT	determiner	<i>a, the</i>	PRP	personal pronoun	<i>I, you, he</i>	WDT	wh-determ.	<i>which, that</i>
EX	existential 'there'	<i>there</i>	PRP\$	possess. pronoun	<i>your, one's</i>	WP	wh-pronoun	<i>what, who</i>
FW	foreign word	<i>mea culpa</i>	RB	adverb	<i>quickly</i>	WP\$	wh-possess.	<i>whose</i>
IN	preposition/ subordin-conj	<i>of, in, by</i>	RBR	comparative adverb	<i>faster</i>	WRB	wh-adverb	<i>how, where</i>
JJ	adjective	<i>yellow</i>	RBS	superlatv. adverb	<i>fastest</i>	\$	dollar sign	<i>\$</i>
JJR	comparative adj	<i>bigger</i>	RP	particle	<i>up, off</i>	#	pound sign	<i>#</i>
JJS	superlative adj	<i>wildest</i>	SYM	symbol	<i>+, %, &</i>	“	left quote	<i>' or “</i>
LS	list item marker	<i>1, 2, One</i>	TO	“to”	<i>to</i>	”	right quote	<i>' or ”</i>
MD	modal	<i>can, should</i>	UH	interjection	<i>ah, oops</i>	(	left paren	<i>[, (, {, <</i>
NN	sing or mass noun	<i>llama</i>	VB	verb base form	<i>eat</i>	)	right paren	<i>],), }, ></i>
NNS	noun, plural	<i>llamas</i>	VBD	verb past tense	<i>ate</i>	,	comma	<i>,</i>
NNP	proper noun, sing.	<i>IBM</i>	VBG	verb gerund	<i>eating</i>	.	sent-end punc	<i>. ! ?</i>
NNPS	proper noun, plu.	<i>Carolinas</i>	VBN	verb past part.	<i>eaten</i>	:	sent-mid punc	<i>: ; ... --</i>

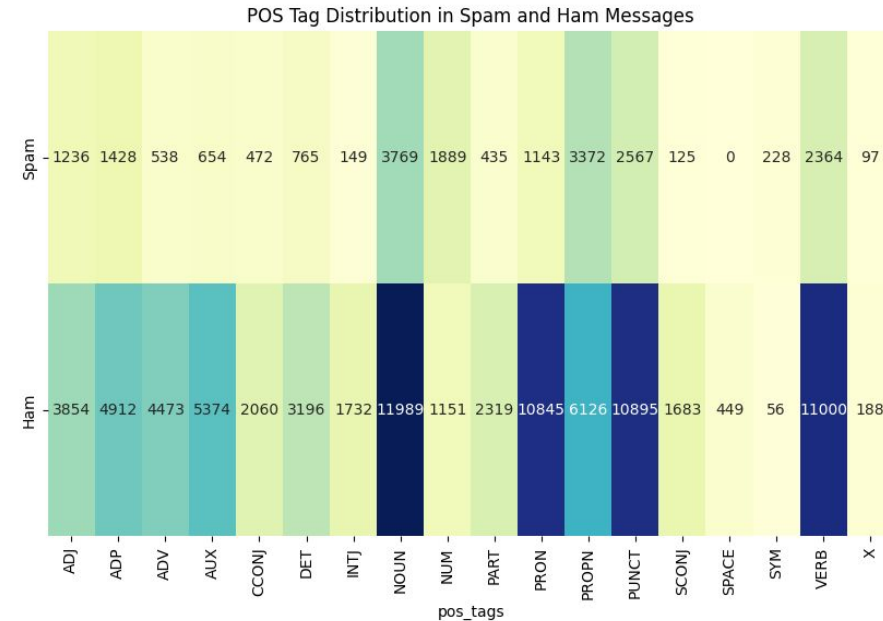
```
for token in nlp(data['message'][0]):
    print(f"Token: {token.text}, POS: {token.pos_}")
```

```
Token: Go, POS: VERB
Token: until, POS: ADP
Token: jurong, POS: PROPN
Token: point, POS: Add code cell
Token: ,, POS: PUN %/Ctrl+M B
Token: crazy, POS: ADJ
Token: .., POS: PUNCT
Token: Available, POS: ADJ
Token: only, POS: ADV
Token: in, POS: ADP
Token: bugis, POS: ADJ
Token: n, POS: CCONJ
Token: great, POS: ADJ
Token: world, POS: NOUN
Token: la, POS: PROPN
Token: e, POS: PROPN
Token: buffet, POS: NOUN
Token: ..., POS: PUNCT
Token: Cine, POS: PROPN
Token: there, POS: ADV
Token: got, POS: VERB
Token: amore, POS: PROPN
Token: wat, POS: NOUN
Token: ..., POS: PUNCT
```

Visualization Analysis: POS Tagging

Key Takeaways:

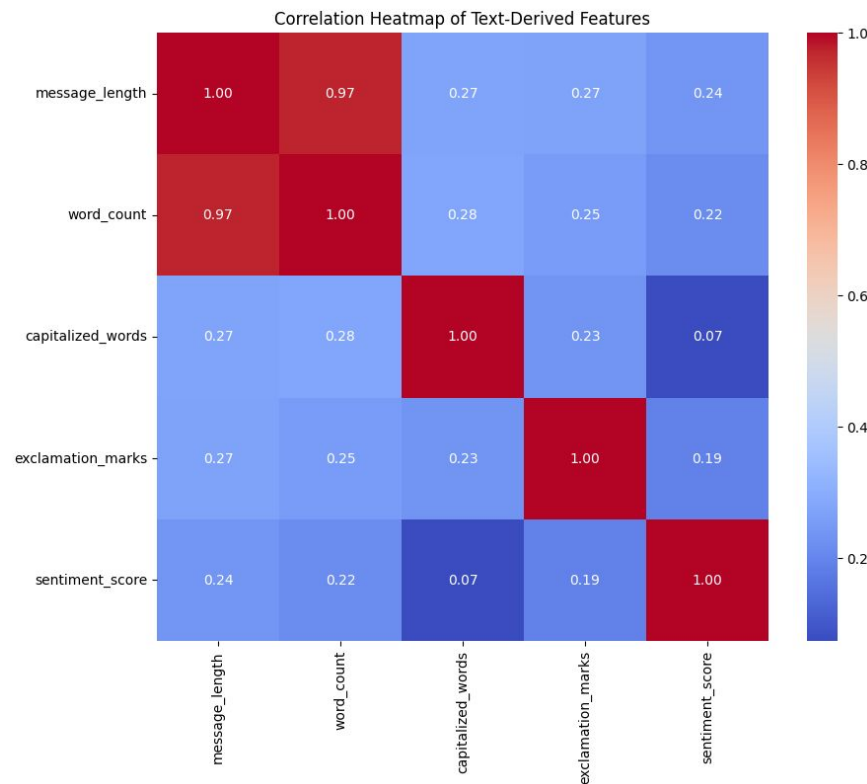
- Spam messages tend to use more structured and promotional language, focusing on nouns and proper nouns, often avoiding conversational pronouns or interjections.
- Ham messages are more personal, descriptive, and conversational, with greater use of pronouns, adjectives, and interjections.



Visualization Analysis: Correlation Analysis

Key Takeaways

1. **Message Length and Word Count:** These features are redundant due to their strong correlation.
2. **Sentiment Score:** Since sentiment does not correlate strongly with structural features, it may provide independent value as a predictor.
3. **Capitalized Words and Exclamation Marks:** These features show some degree of correlation with message length but may still provide useful information.



Dimensionality Analysis

Text data, when vectorized, often has thousands (or even millions) of features (e.g., unique words, n-grams).

To solve this problem, we can apply the same dimensionality reduction techniques seen in the previous section in addition to removing stop words, low-frequency words, and low-correlation words.

```
Original number of words: 8404  
Number of low-frequency words removed: 6761  
Number of words after filtering: 1643
```

EDA For Time Series



Time Series: Air Quality Data

This dataset contains the responses of a gas multisensor device deployed on the field in an Italian city. Hourly responses averages are recorded along with gas concentrations references from a certified analyzer.

```
from ucimlrepo import fetch_ucirepo
import pandas as pd

# Fetch dataset
air_quality = fetch_ucirepo(id=360)

# Convert features and targets to pandas DataFrame
data = pd.DataFrame(air_quality.data.features, columns=air_quality.features)

data.head()
```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
0	3/10/2004	18:00:00	2.6	1360	150	11.9	1046	166	1056	113	1692	1268	13.6	48.9	0.7578
1	3/10/2004	19:00:00	2.0	1292	112	9.4	955	103	1174	92	1559	972	13.3	47.7	0.7255
2	3/10/2004	20:00:00	2.2	1402	88	9.0	939	131	1140	114	1555	1074	11.9	54.0	0.7502
3	3/10/2004	21:00:00	2.2	1376	80	9.2	948	172	1092	122	1584	1203	11.0	60.0	0.7867
4	3/10/2004	22:00:00	1.6	1272	51	6.5	836	131	1205	116	1490	1110	11.2	59.6	0.7888



Air Quality Dataset Features

Feature	Description
Date	Date of the observation (format: YYYY-MM-DD).
Time	Time of the observation (format: HH:MM:SS).
CO(GT)	Concentration of Carbon Monoxide (mg/m^3).
PT08.S1(CO)	Sensor response for CO (likely in arbitrary units or volts).
NMHC(GT)	Non-methane hydrocarbons concentration ($\mu\text{g}/\text{m}^3$).
C6H6(GT)	Benzene concentration ($\mu\text{g}/\text{m}^3$).
PT08.S2(NMHC)	Sensor response for NMHC (arbitrary units or volts).
NOx(GT)	Nitrogen oxides concentration (ppb or $\mu\text{g}/\text{m}^3$).
PT08.S3(NOx)	Sensor response for NOx (arbitrary units or volts).
NO2(GT)	Nitrogen dioxide concentration (ppb or $\mu\text{g}/\text{m}^3$).
PT08.S4(NO2)	Sensor response for NO2 (arbitrary units or volts).
PT08.S5(O3)	Sensor response for Ozone (arbitrary units or volts).
T	Temperature ($^{\circ}\text{C}$).
RH	Relative humidity (%).
AH	Absolute humidity (g/m^3).



Frequency of Recordings

In this dataset, we have two columns that represent timestamps:

- **Date:** Records Month/Day/Year: 391 different days
- **Time:** Recoding the hour: 24, 15, and 6 hour recordings

```
# Count the number for each date
number_of_hours_recorded_for_each_day = []
for date in data['Date'].unique():
    count = len(data[data['Date'] == date])
    number_of_hours_recorded_for_each_day.append(count)

print(f"The different number of hours recorded for each day is: {set(number_of_hours_recorded_for_each_day)}")

The different number of hours recorded for each day is: {24, 6, 15}
```

```
print(data['Date'].unique())
```

```
['3/10/2004' '3/11/2004' '3/12/2004' '3/13/2004' '3/14/2004' '3/15/2004'
 '3/16/2004' '3/17/2004' '3/18/2004' '3/19/2004' '3/20/2004' '3/21/2004'
 '3/22/2004' '3/23/2004' '3/24/2004' '3/25/2004' '3/26/2004' '3/27/2004'
 '3/28/2004' '3/29/2004' '3/30/2004' '3/31/2004' '4/1/2005' '4/2/2005'
 '4/3/2005' '4/4/2005']
```

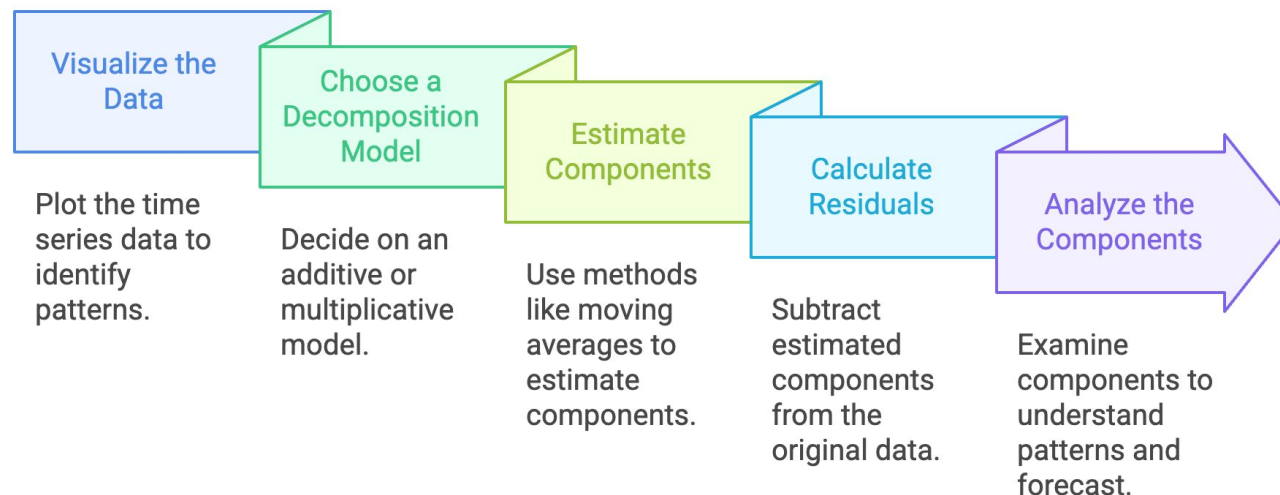
391

Date	Time
3/10/2004	18:00:00
3/10/2004	19:00:00
3/10/2004	20:00:00
3/10/2004	21:00:00
3/10/2004	22:00:00

Visualization Analysis: Decomposition Analysis

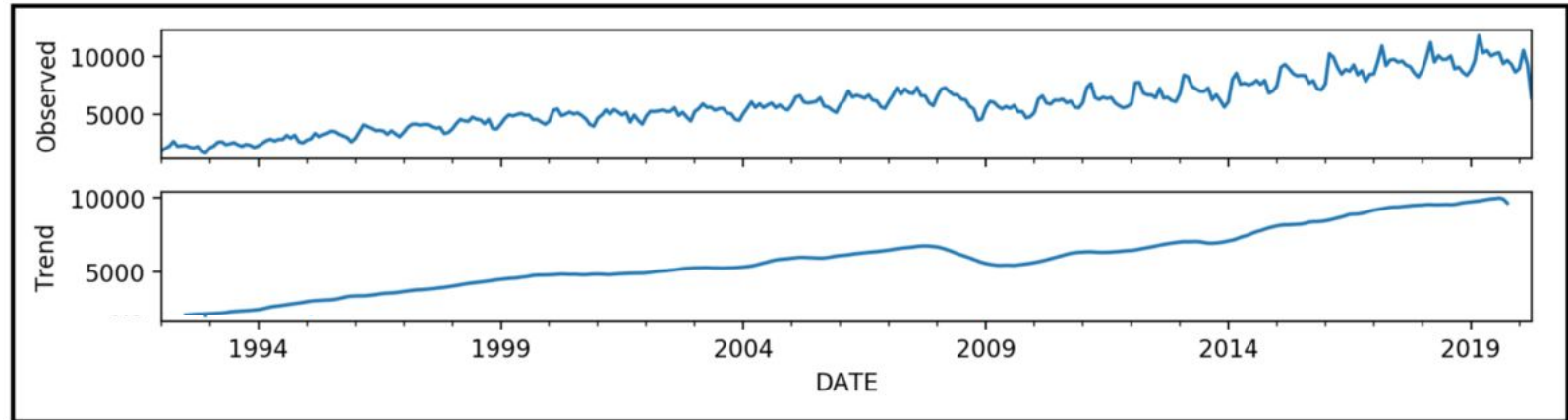
For time series data, we start exploring the data through visualization analysis because the pattern of the data dictates the rest of the analysis and pre-processing.

Steps in Decomposition Analysis



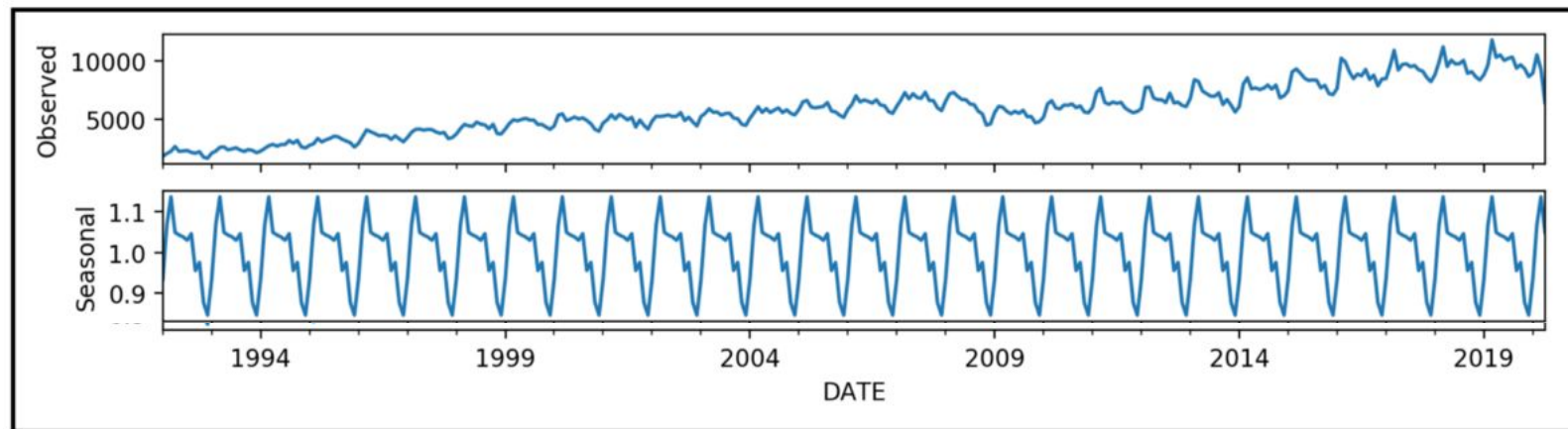
Decomposition Analysis: Types of Patterns

Trend: The trend is the long-term movement or direction in the time series. It reflects persistent increases or decreases over a long period, ignoring short-term fluctuations.



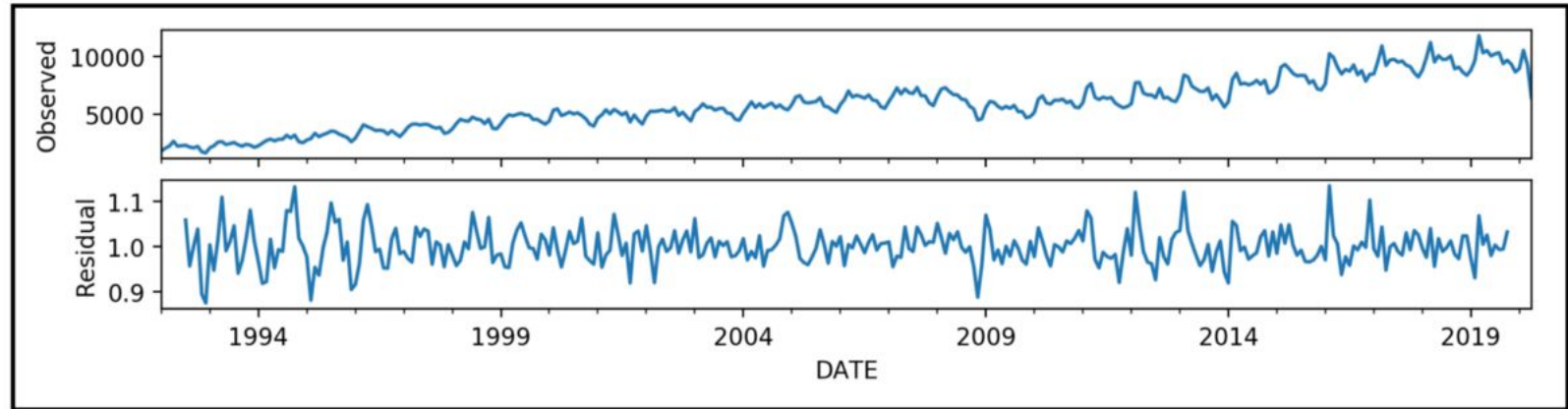
Decomposition Analysis: Types of Patterns

Seasonality: Refers to regular, repeating patterns in the time series over specific intervals (e.g., daily, weekly, yearly). It captures cyclical behaviors that recur predictably over time.



Decomposition Analysis: Types of Patterns

Residual: Residuals represent the noise or the unexplained variability in the time series after removing the trend and seasonality. They are the random, unpredictable components of the data.



Decomposition Analysis: Identify Patterns

It is a best practice to explore the trends of all features in the dataset. To avoid redundancy, we will take the CO(GT) as the feature of choice for the rest of the analysis.

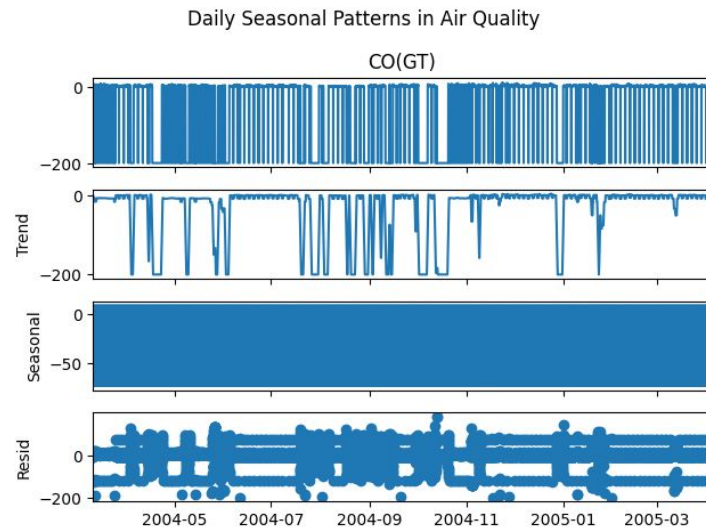
To identify patterns, we will use different time periods to analyze:

```
periods = {  
    'Daily': 24,           # 24 hours in a day  
    'Weekly': 24 * 7,      # 168 hours in a week  
    'Monthly': 24 * 30,    # ~720 hours in a month  
    'Annual': 24 * 365     # 8760 hours in a year  
}
```

Decomposition Analysis: Identify Patterns

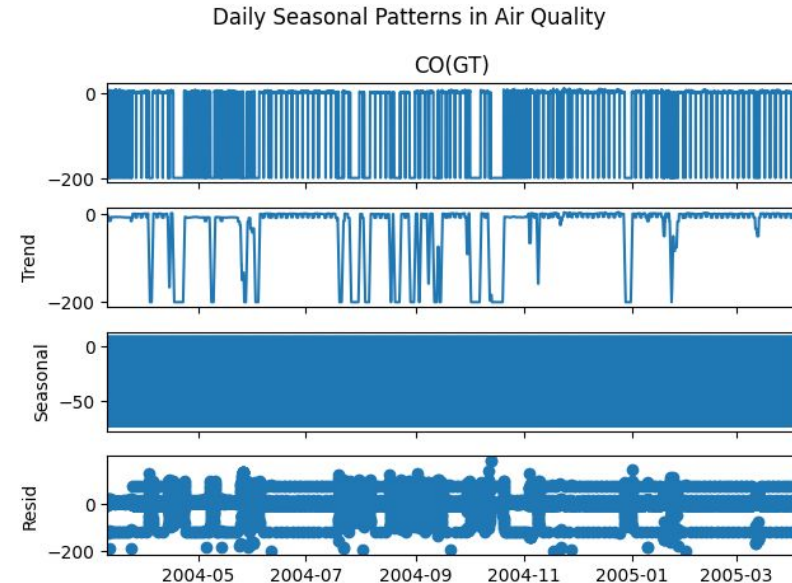
For Daily periods, we can tell that:

- **Original:** There is substantial daily variation in CO levels throughout the measured period from 2004 to 2005.
- **Trend:** There are periods where the baseline CO level suddenly drops (those sharp downward spikes) before returning to normal levels. This might indicate temporary changes in local conditions or measurement anomalies.



Decomposition Analysis: Identify Patterns

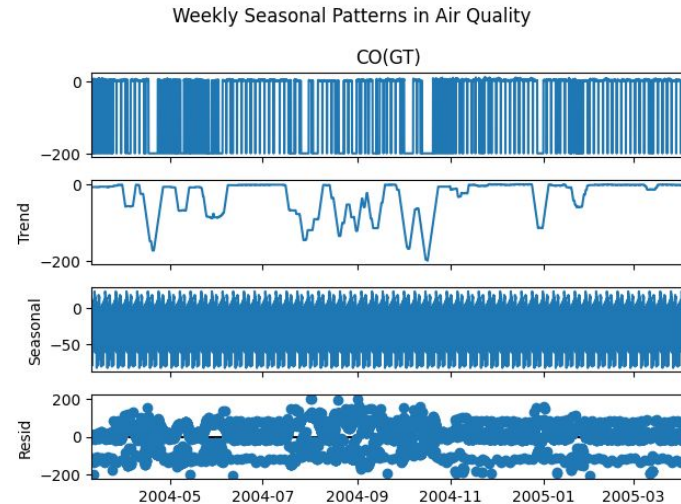
- **Seasonality:** The flat, uniform pattern suggests that CO follows a very regular daily cycle. The negative values indicate that CO levels tend to decrease during certain parts of the day. This makes sense from an air quality perspective, as CO emissions often follow human activity patterns - higher during peak traffic hours and lower during nighttime.
- **Resid:** The scattered points indicate some random variation in CO levels that can't be explained by daily patterns or long-term trends. We see that these residuals are mostly clustered around zero with some outliers, suggesting our seasonal decomposition has captured the main patterns well.



Decomposition Analysis: Identify Patterns

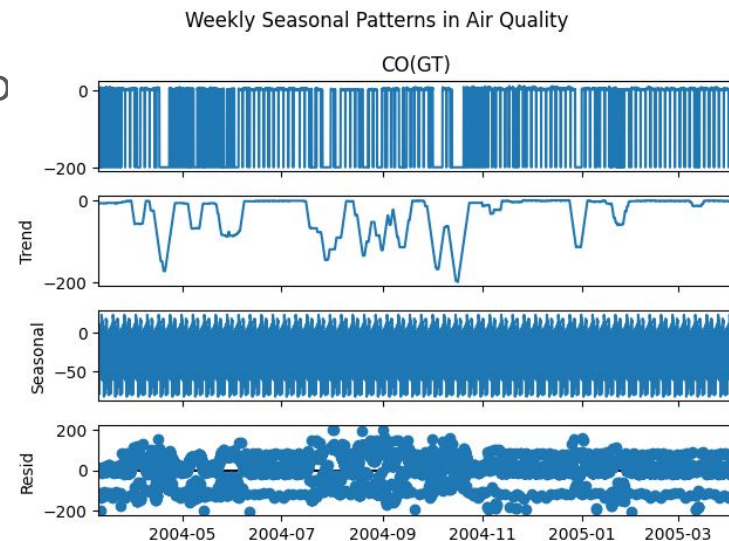
For weekly analysis:

- **Original:** Just like in the daily analysis, we see the raw CO measurements showing regular fluctuations. The pattern of variation remains consistent, helping us confirm that the data quality is maintained throughout the collection period.
- **Trend:** We can observe smoother, longer-term variations compared to the daily analysis. These fluctuations might be related to broader weather patterns or seasonal changes that affect CO levels over weeks rather than days. The trend line shows several notable dips, particularly around late 2004, which could indicate periods of improved air quality or changes in local emission sources.



Decomposition Analysis: Identify Patterns

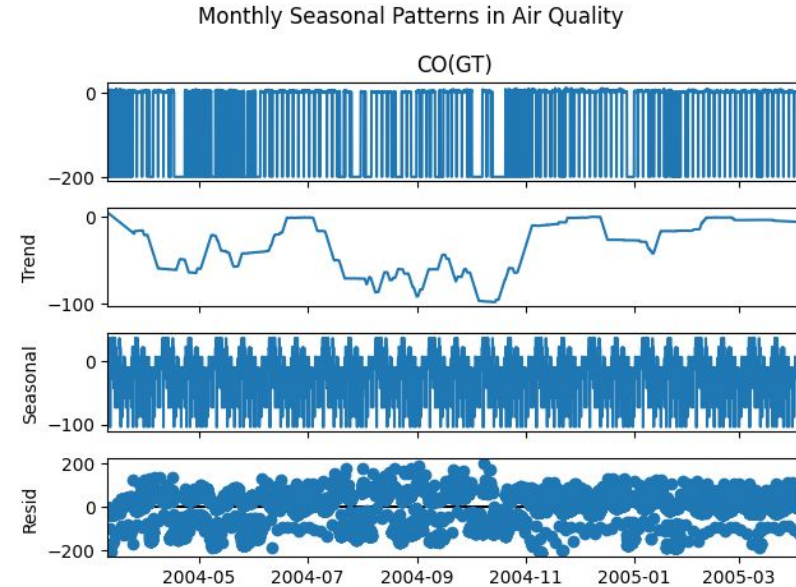
- **Seasonal:** The regular up-and-down pattern suggests that CO levels follow a consistent weekly rhythm. The peaks and troughs likely correspond to weekday versus weekend patterns. The pattern repeats very consistently, suggesting these weekly cycles are quite stable.
- **Resid:** The residuals appear fairly well-distributed around zero, though with some notable outliers. This suggests that while our weekly decomposition captures the main patterns well, there are still some unexplained variations in CO levels that might be worth investigating.



Decomposition Analysis: Identify Patterns

For monthly analysis, we notice:

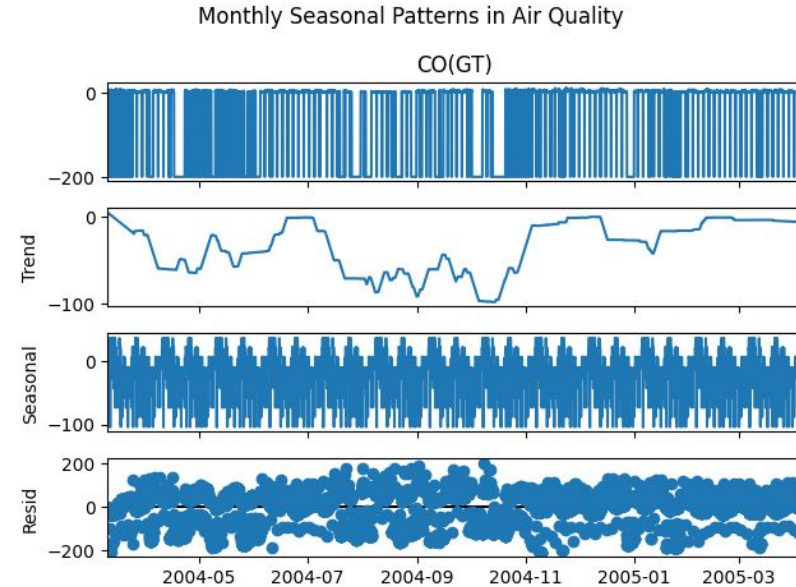
- **Original:** Consistent with what we saw in our daily and weekly analyses, showing the base measurements over the entire period, which validates the reliability of the data collection.
- **Trend:** There's a noticeable dip in CO levels during the latter part of 2004, reaching its lowest point around October-November. The levels seem to recover and stabilize somewhat in early 2005.



Decomposition Analysis: Identify Patterns

For monthly analysis, we notice:

- **Seasonal:** The monthly seasonal pattern appears more irregular and complex. We see more variable amplitudes in the seasonal fluctuations. The pattern suggests that CO levels change significantly month to month.
- **Resid:** The residuals appear fairly well distributed around zero. There are some notable outliers, suggesting occasional unusual events affecting air quality.

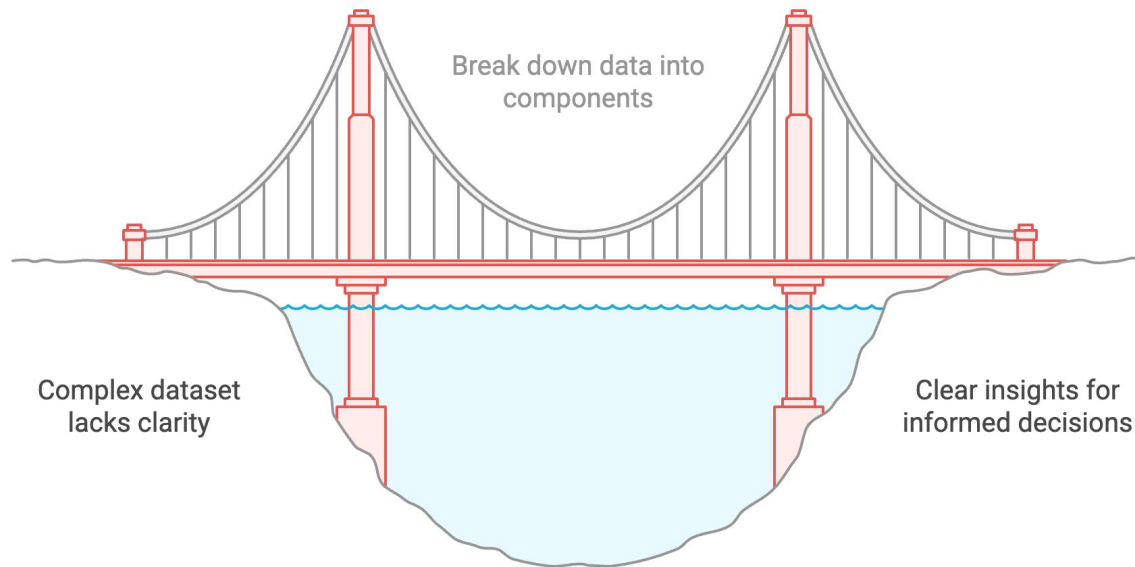


Decomposition Analysis: Decomposition Models



AMERICAN
UNIVERSITY
OF BEIRUT

Enhance Forecasting Accuracy with Decomposition



Decomposition Analysis: Decomposition Models

Time series decomposition models break down a time series into several basic components that, when combined, recreate the original data. There are two main types of decomposition models:

1. **Additive Decomposition:** This model assumes that the components add together to make the original series: $Y(t) = \text{Trend}(t) + \text{Seasonal}(t) + \text{Residual}(t)$
2. **Multiplicative Decomposition:** This model assumes the components multiply together: $Y(t) = \text{Trend}(t) \times \text{Seasonal}(t) \times \text{Residual}(t)$

Decomposition Analysis: Decomposition Models

To Choose a decomposition model, you need to analyze how the different patterns interact:

1. Are the seasonal variations relatively constant throughout the series (suggesting additive)?
2. Do the variations seem to grow or shrink with the overall level (suggesting multiplicative)?
3. Are there any negative values? (If yes, multiplicative decomposition might not be appropriate)

Decomposition Analysis: Decomposition Models

Looking at our previous decomposition plots across different time scales (daily, weekly, and monthly), we can identify several key characteristics that help us choose between additive and multiplicative decomposition:

1. We saw that the **seasonal variations** maintained relatively **consistent amplitudes** throughout the observation period. The fluctuations didn't grow or shrink significantly as the overall CO levels changed. This is a strong indicator that an **additive model** would be more appropriate.
2. Our data contained **negative values**, as we could see in the original time series plots. Since multiplying by negative seasonal factors would invert the pattern, this favors an **additive model**.

Decomposition Analysis: Decomposition Models

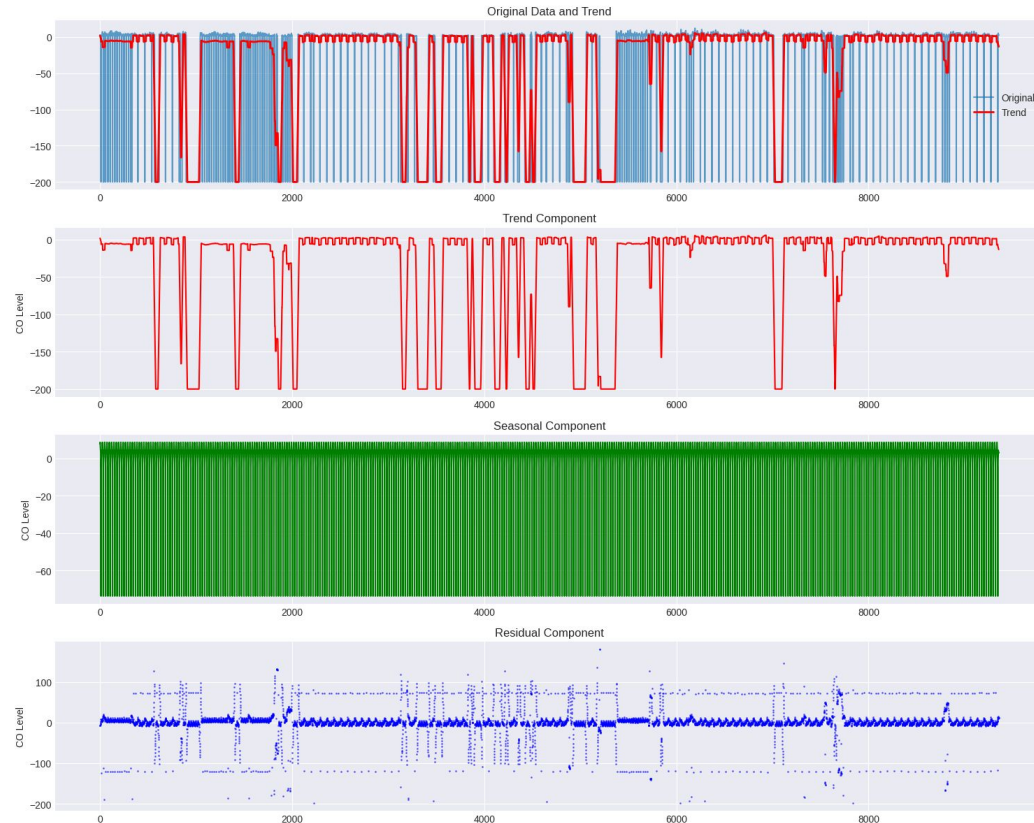
3. In our decomposition plots, **the residuals** appeared to have fairly consistent **spread around zero**, without showing a fan-shaped pattern (where spread increases with the level). This consistent variance is another characteristic that supports using an additive model.

4. The **seasonal patterns** we observed didn't appear to scale up or down with the trend level. For example, when the trend showed dips or rises, the magnitude of daily and weekly patterns remained stable. This independence between trend and seasonal variation is a classic indicator that **additive decomposition** is more appropriate.

Decomposition Analysis: Decomposition Models

Now that we have chosen the additive model, we can implement it and extract some statistics from our new data model.

```
# Perform additive decomposition
decomposition = seasonal_decompose(
    df[column_name],
    period=period,
    model='additive',
    extrapolate_trend='freq'
)
```

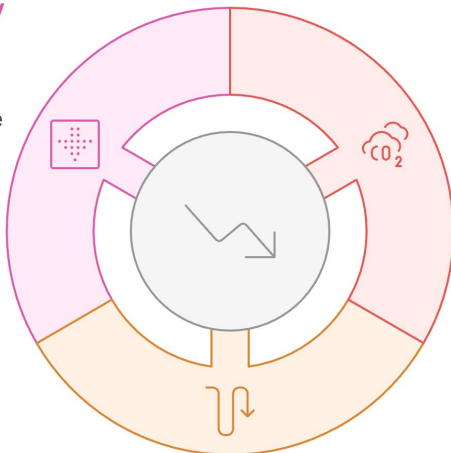


Statistical Analysis: Component Statistics

CO Level Trend Analysis

Negative Territory

Suggests CO levels predominantly remain in negative values



Average CO Level

Indicates a negative average baseline CO level

High Variation

Shows substantial variation in CO levels over time

Component Statistics:

Trend Component:

Mean: -34.20

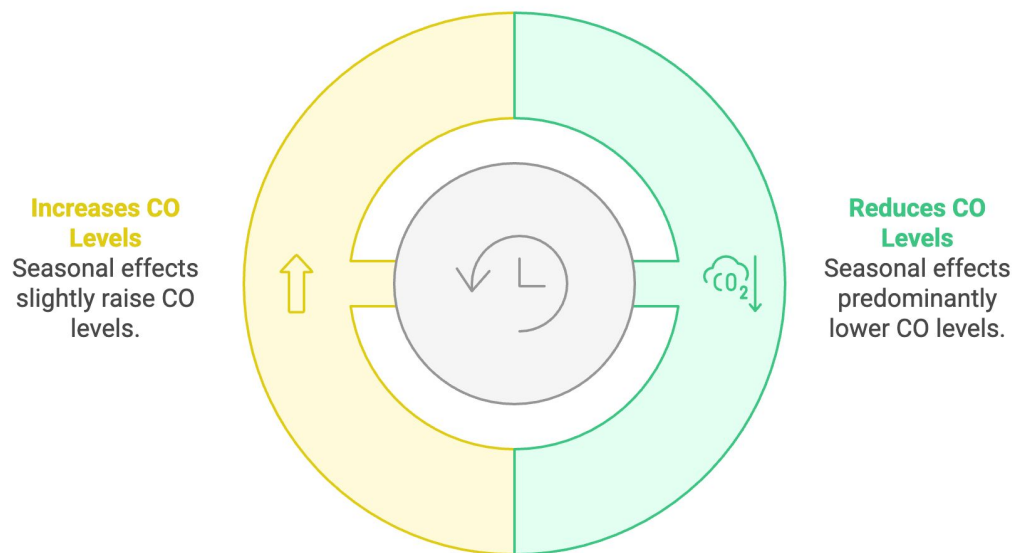
Std: 66.52

Min: -200.00

Max: 5.59

Statistical Analysis: Component Statistics

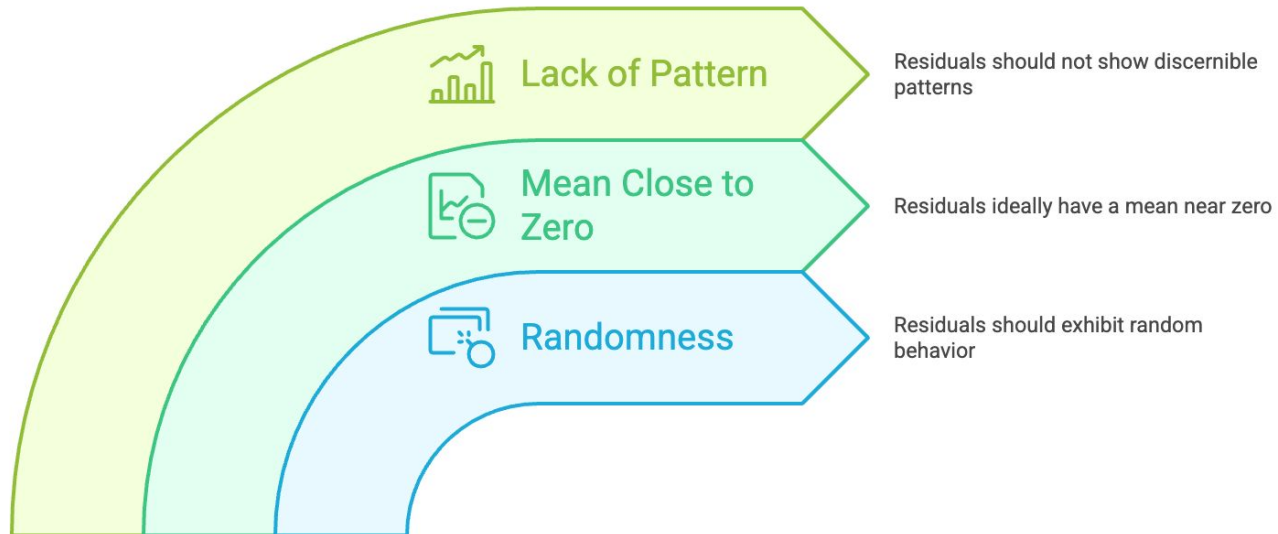
Seasonal Effects on CO Levels



Seasonal Component:
Mean: -0.00
Std: 15.72
Min: -73.66
Max: 8.66

Statistical Analysis: Calculate Residuals

The calculate residuals step in time series decomposition involves isolating the unpredictable or unexplained part of the data, which remains after removing the trend and seasonality components.

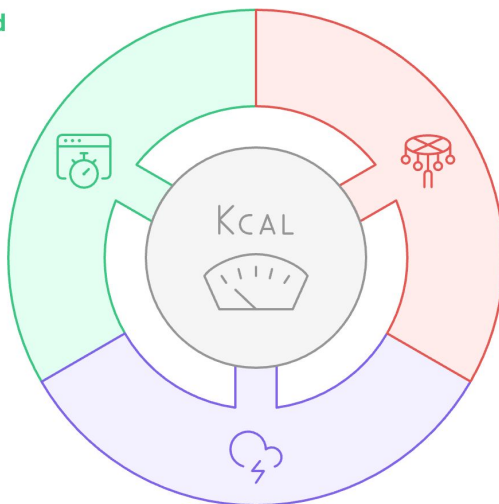


Statistical Analysis: Component Statistics

Residual Component Analysis

Systematic Patterns Captured

Trends and
seasonals
accounted for
effectively



Large Swings

Significant
fluctuations in CO
levels not
explained by
patterns

Irregular Events

Unpredictable
factors impacting
CO levels

Residual Component:
Mean: -0.00
Std: 31.61
Min: -198.47
Max: 180.37

Statistical Analysis: Component Statistics

Putting it all together, this decomposition reveals that:

1. The air quality measurements generally stay below the reference point (negative trend)
2. Regular patterns (seasonal effects) are present but relatively modest compared to both the trend and irregular variations
3. There are substantial irregular variations that can cause CO levels to spike up or down dramatically

EDA On Image Data

Image Dataset: COCO Dataset

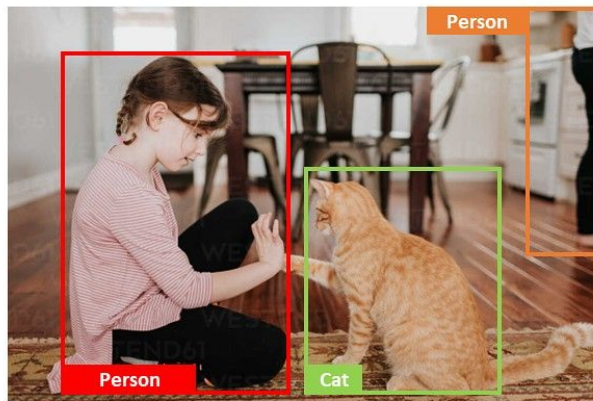
The COCO (Common Objects in Context) dataset is a large-scale object detection, segmentation, and recognition dataset.

Recognition



Dog

Object detection

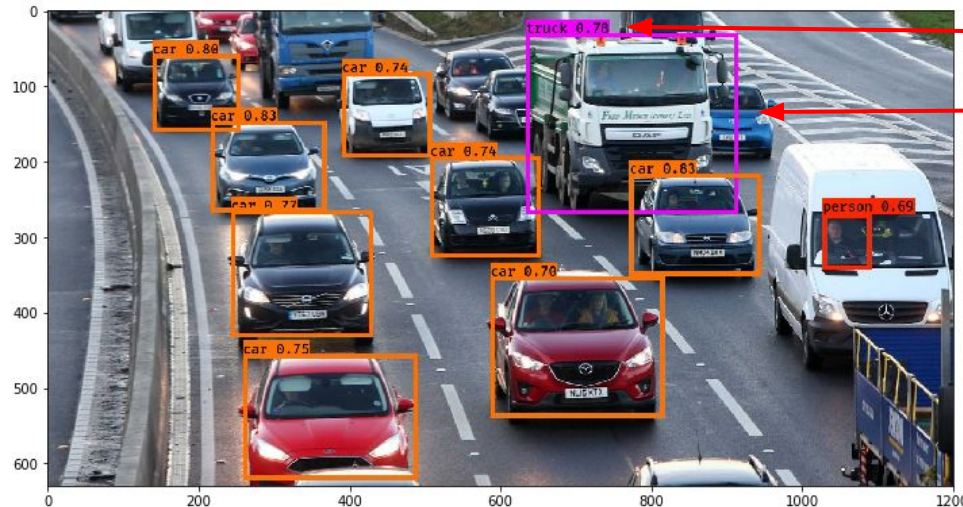


Segmentation



Load COCO Dataset

For our image dataset, we will use the `instances_train2017` annotations, which are **object detection** annotations. This means that they contain a label that tells where the **bounding boxes** exist in our dataset, as well as **class** that the box relates to.



Class

Bounding Box



COCO Dataset Images Annotations

We will select 50 images with their annotations. This resulted in 404 annotations which belong to 58 different categories.

The number of annotations is much higher than the number of images because each annotation (bounding box) in the image has its own annotation. In other words, one image has multiple annotations and one annotation refers to one image.

```
Selected number of images: 50  
Selected number of annotations: 404  
Selected number of categories: 58
```

COCO Dataset: Selected Categories

```
unique_categories = {}
for category in selected_categories:
    if category['supercategory'] not in unique_categories:
        unique_categories[category['supercategory']] = []
    if category['name'] not in unique_categories[category['supercategory']]:
        unique_categories[category['supercategory']].append(category['name'])

for super_catrgory in unique_categories:
    print(f"{super_catrgory}: {unique_categories[super_catrgory]}")

person: ['person']
vehicle: ['car', 'motorcycle', 'airplane', 'bus', 'train', 'truck', 'boat']
outdoor: ['traffic light', 'fire hydrant', 'stop sign', 'bench']
animal: ['bird', 'cat', 'dog', 'elephant', 'giraffe']
accessory: ['backpack', 'umbrella', 'handbag', 'tie', 'suitcase']
sports: ['skis', 'snowboard', 'sports ball', 'kite', 'baseball glove', 'skateboard', 'surfboard']
kitchen: ['bottle', 'cup', 'fork', 'knife', 'spoon', 'bowl']
food: ['apple', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut', 'cake']
furniture: ['chair', 'couch', 'potted plant', 'dining table']
electronic: ['cell phone']
appliance: ['microwave', 'oven', 'toaster', 'sink', 'refrigerator']
indoor: ['book', 'clock', 'vase', 'teddy bear', 'hair drier', 'toothbrush']
```

Statistical Analysis

1. Pixel Intensity Analysis

- a. Pixel Statistics: Compute pixel statistics across the entire dataset or per channel (e.g., RGB).
- b. Skewness and Kurtosis: Understand asymmetry and peakedness in pixel intensities.

2. Image Dimensions and Aspect Ratios

- a. Image Size Distribution: Evaluate the range of image dimensions.
- b. Aspect Ratio Distribution: Detect common and outlier aspect ratios.

3. Color Composition

- a. Color Channel Correlations: Identify relationships between RGB channels.
- b. Color Diversity: Analyze the variety of colors using metrics like Shannon entropy.

4. Quality Metrics:

- a. Sharpness: Evaluate edge gradients to measure sharpness.
- b. Noise Levels: Quantify noise using metrics like PSNR or SNR.
- c. Blurriness Detection: Analyze variance of Laplacian to detect blur.

Statistical Analysis: Pixel Intensity Analysis

After running Pixel Intensity Analysis on our dataset, we got the following results:

```
def pixel_intensity_analysis(selected_images):
    means, stds, skews, kurtoses = [], [], [], []
    intensity_distributions = {'R': [], 'G': [], 'B': []}

    for image_info in selected_images:
        # Load image
        image = imread(image_info['coco_url']) # Read directly from coco_url
        if image is not None:
            # Compute per-channel statistics
            for i, channel in enumerate(['R', 'G', 'B']):
                channel_data = image[:, :, i].flatten()
                intensity_distributions[channel].extend(channel_data)
                means.append(np.mean(channel_data))
                stds.append(np.std(channel_data))
                skews.append(skew(channel_data))
                kurtoses.append(kurtosis(channel_data))
```

```
# Summary statistics
print(f"Mean pixel values (R, G, B): {np.mean(means, axis=0)}")
print(f"Std pixel values (R, G, B): {np.mean(stds, axis=0)}")
print(f"Skewness (R, G, B): {np.mean(skews, axis=0)}")
print(f"Kurtosis (R, G, B): {np.mean(kurtoses, axis=0)}")
```

```
pixel_intensity_analysis(selected_images)
```

```
Mean pixel values (R, G, B): 120.92563869315602
Std pixel values (R, G, B): 64.71334365657572
Skewness (R, G, B): 0.12161625863878894
Kurtosis (R, G, B): -0.1807936348876289
```

Statistical Analysis: Pixel Intensity Analysis

1. Mean Pixel Values: 120.93

The average pixel intensity is around the mid-range (0-255 scale), indicating that the dataset has **balanced brightness** and no extreme overexposure or underexposure.

2. Standard Deviation (Std): 64.71

A relatively high (the maximum possible standard deviation occurs when pixel values are evenly split between these extremes (0 and 255), which would yield a standard deviation of approximately 73.61) standard deviation suggests that the pixel values are well-spread across a wide range of intensities, indicating **good contrast** within the images.



Statistical Analysis: Pixel Intensity Analysis

3. Skewness measures the asymmetry of a probability distribution. It indicates whether the data is skewed towards the left (negative skew) or right (positive skew) of the mean.
4. Kurtosis shows how sharp or flat a graph is and how many extreme values it has. A high kurtosis means more extreme values, while a low kurtosis means fewer extreme values.

$$S = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^3}{\sigma^3}$$

- S : Skewness
- n : Number of data points
- x_i : Pixel intensity value
- μ : Mean of the data
- σ : Standard deviation of the data

$$K = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^4}{\sigma^4} - 3$$

- K : Kurtosis
- n : Number of data points
- x_i : Pixel intensity value
- μ : Mean of the data
- σ : Standard deviation of the data
- -3 : Adjusts for comparison with a normal distribution (excess kurtosis).

Statistical Analysis: Pixel Intensity Analysis

1. The images in this dataset may have uniform lighting or low contrast, as pixel values are concentrated around a central intensity.
2. Such datasets could require normalization or contrast enhancement for better model performance.

Pixel Intensity Distribution Analysis

Peaked but symmetric distribution
Symmetrical with a pronounced peak.

Low Skewness

Flat and symmetric distribution

Evenly spread with no sharp peaks.

High Kurtosis

Highly skewed and peaked distribution

Distribution with extreme values and sharp peak.

High Skewness

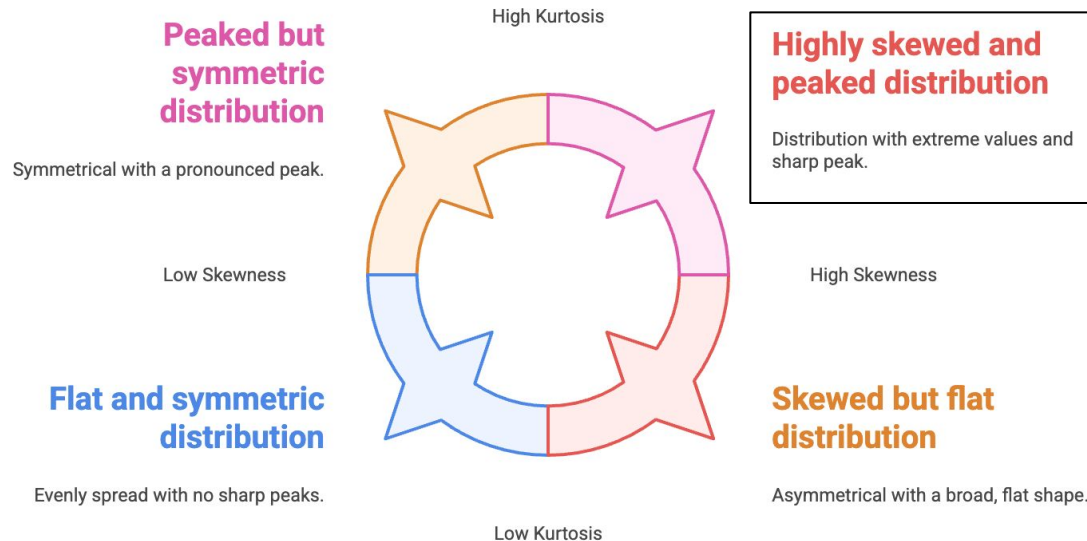
Skewed but flat distribution

Asymmetrical with a broad, flat shape.

Low Kurtosis

Statistical Analysis: Pixel Intensity Analysis

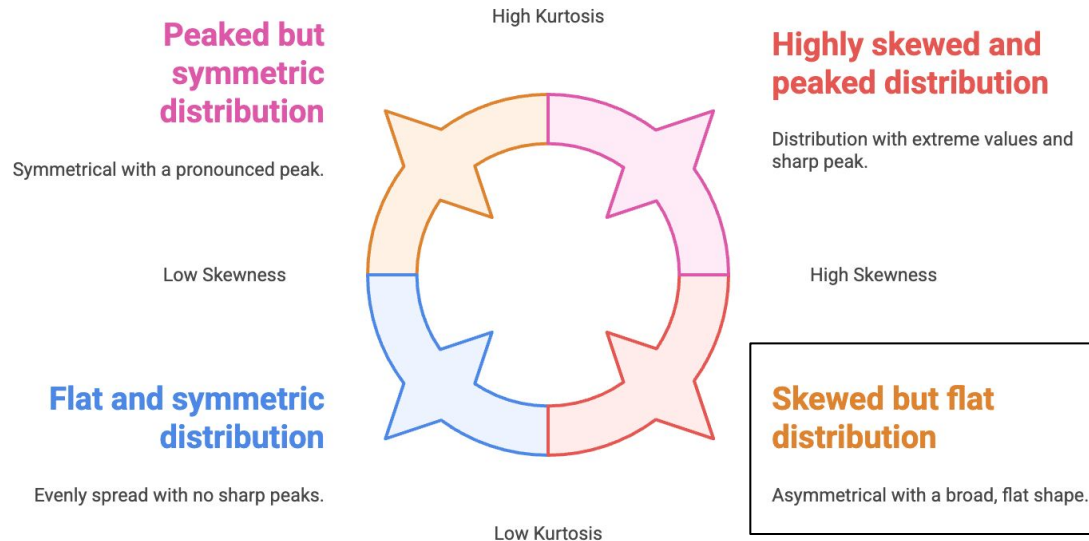
Pixel Intensity Distribution Analysis



1. The dataset likely contains images with strong shadows, highlights, or unbalanced brightness.
2. Data augmentation techniques like histogram equalization or adaptive brightness adjustments may help balance the dataset.

Statistical Analysis: Pixel Intensity Analysis

Pixel Intensity Distribution Analysis

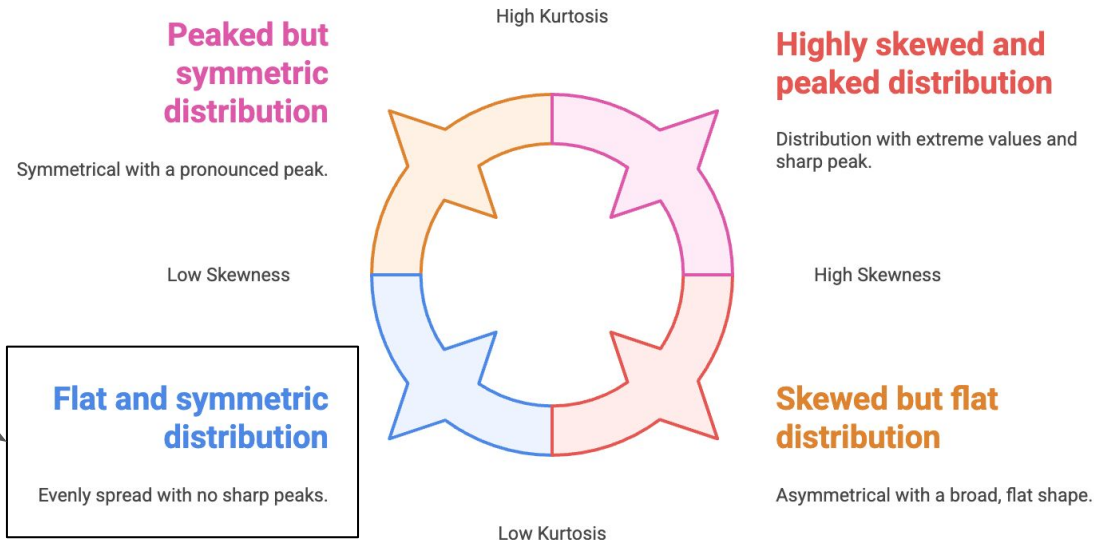


1. This indicates uneven intensity distribution, often due to complex lighting or varied exposure conditions across the dataset.
2. Requires careful preprocessing to ensure uniformity, such as channel-wise normalization or exposure correction.
3. It might also point to a diverse dataset, which can be advantageous for generalization.

Statistical Analysis: Pixel Intensity Analysis

Pixel Intensity Distribution Analysis

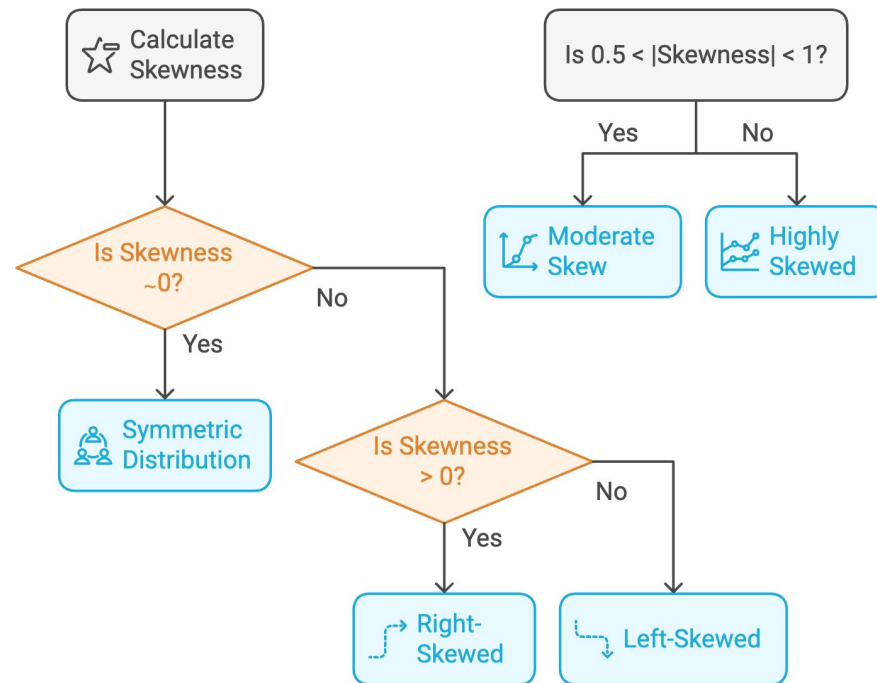
1. Likely represents images with balanced lighting and a wide range of pixel intensities.
2. Such distributions are ideal as they indicate good contrast and no overexposure or underexposure.
3. Preprocessing may still include standard normalization but requires minimal correction.



Statistical Analysis: Pixel Intensity Analysis

Skewness: 0.121

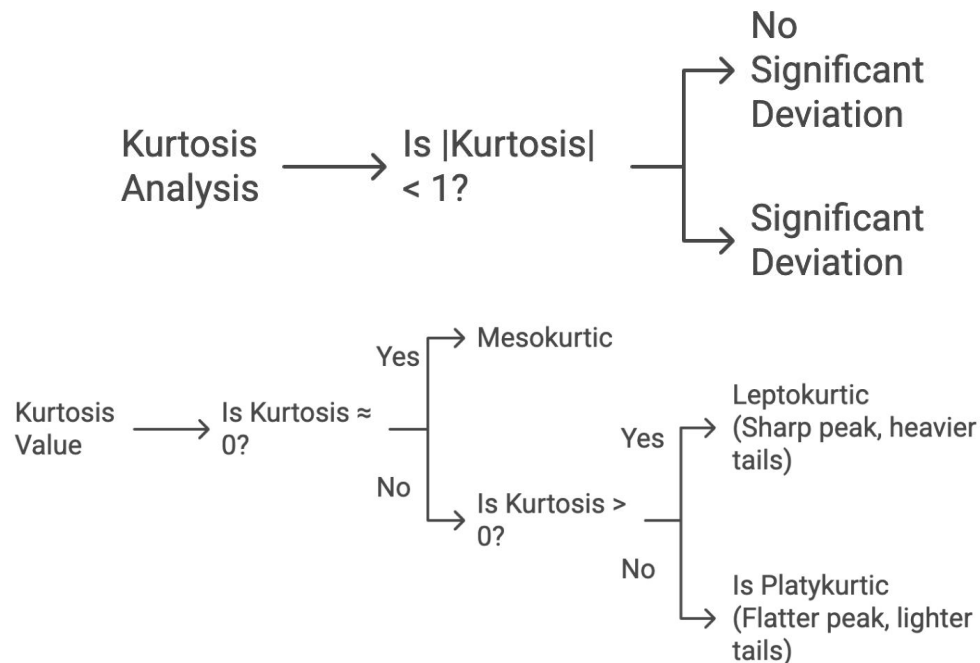
The average pixel intensity is around the mid-range (0-255 scale), indicating that the dataset has balanced brightness and no extreme overexposure or underexposure.



Statistical Analysis: Pixel Intensity Analysis

Kurtosis: -0.187

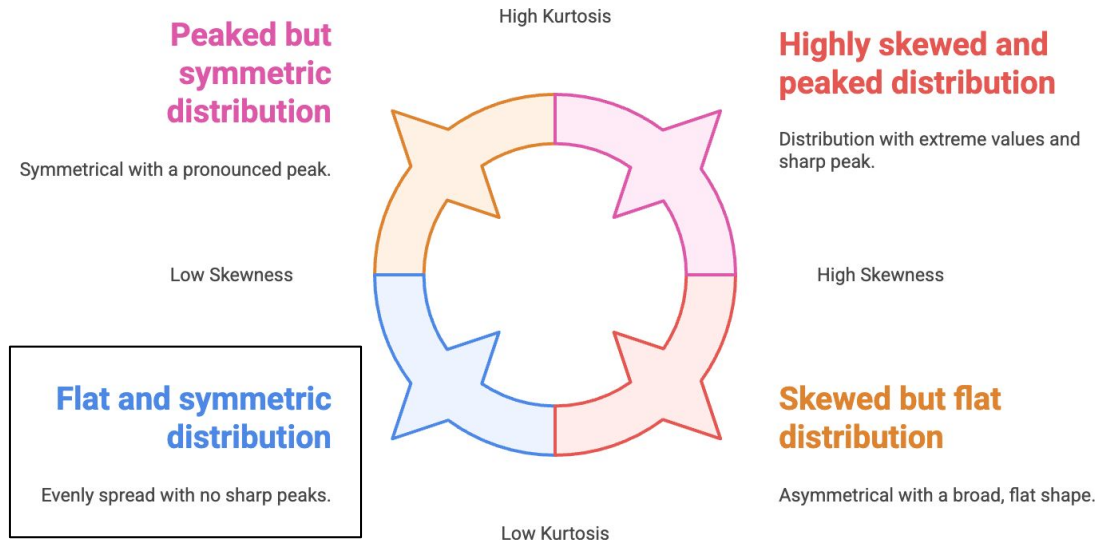
A slightly negative kurtosis indicates a slightly flatter distribution (platykurtic), meaning there are fewer extreme outliers in pixel intensities compared to a normal distribution.



Statistical Analysis: Pixel Intensity Analysis

Our dataset falls within the 'flat and symmetric' distribution.

Pixel Intensity Distribution Analysis



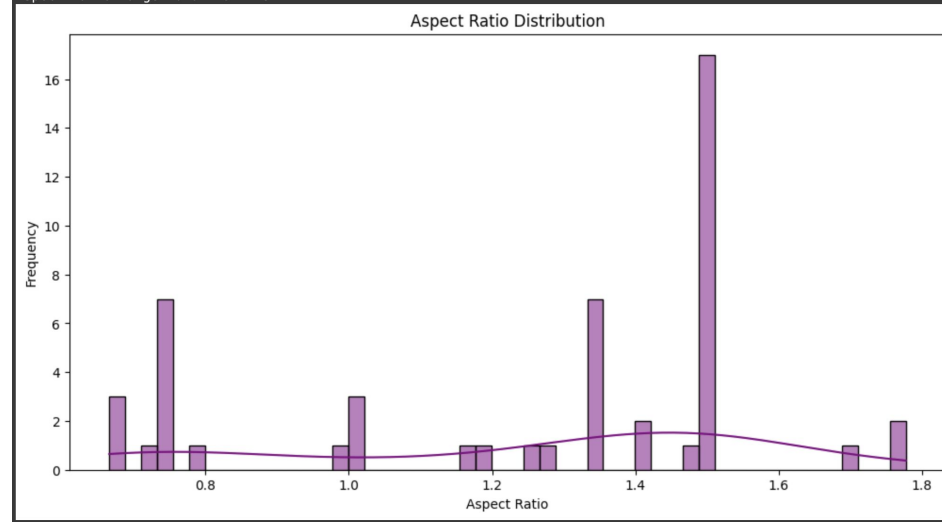
Statistical Analysis: Image Dimensions

The aspect ratio is calculated as width/height.

The range indicates a mix of:

1. Portrait-oriented images (aspect ratio <1)
2. Square-like images (aspect ratio ≈ 1)
3. Landscape-oriented images (aspect ratio >1)

Image height range: 360 to 640
Image width range: 375 to 640
Aspect ratio range: 0.67 to 1.78



Statistical Analysis: Image Dimensions

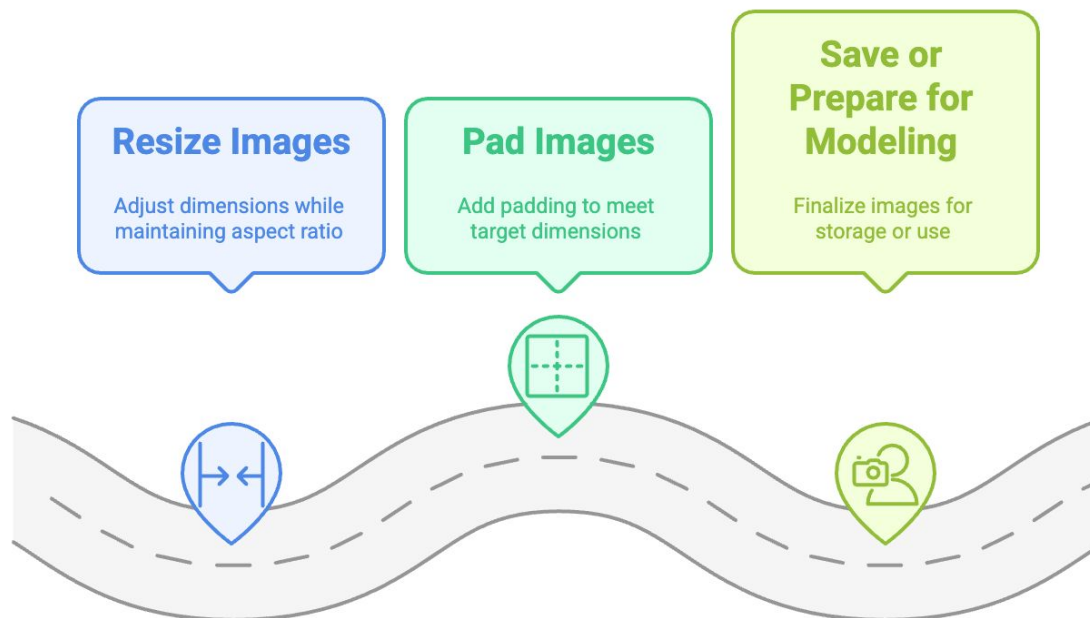
The aspect ratio is calculated as width/height.

The range indicates a mix of:

1. Portrait-oriented images (aspect ratio <1)
2. Square-like images (aspect ratio ≈ 1)
3. Landscape-oriented images (aspect ratio >1)

Statistical Analysis: Image Dimensions

We need to pre-process our images to be of the same size since a machine learning model will expect a fixed size as an input.





Statistical Analysis: Image Dimensions

```
def preprocess_image(image, target_size=(256, 256)):  
    # Load image  
    h, w, c = image.shape  
  
    # Resize while keeping aspect ratio  
    scale = min(target_size[0] / h, target_size[1] / w)  
    new_h, new_w = int(h * scale), int(w * scale)  
    resized_image = cv2.resize(image, (new_w, new_h), interpolation=cv2.INTER_AREA)  
  
    # Pad to target size  
    top = (target_size[0] - new_h) // 2  
    bottom = target_size[0] - new_h - top  
    left = (target_size[1] - new_w) // 2  
    right = target_size[1] - new_w - left  
  
    padded_image = cv2.copyMakeBorder(  
        resized_image, top, bottom, left, right, cv2.BORDER_CONSTANT, value=[0, 0, 0]  
    )  
  
    return padded_image
```



Statistical Analysis: Image Dimensions

We have run into the problem of having annotations with coordinates that no longer match their images, so we need to process the annotations as well.

```
# Update annotations
updated_annotations = []
for ann in annotations:
    # Extract original bounding box
    x_min, y_min, width, height = ann["bbox"]
    x_max = x_min + width
    y_max = y_min + height

    # Scale coordinates
    x_min = x_min * scale
    x_max = x_max * scale
    y_min = y_min * scale
    y_max = y_max * scale

    # Add padding offsets
    x_min += left
    x_max += left
    y_min += top
    y_max += top

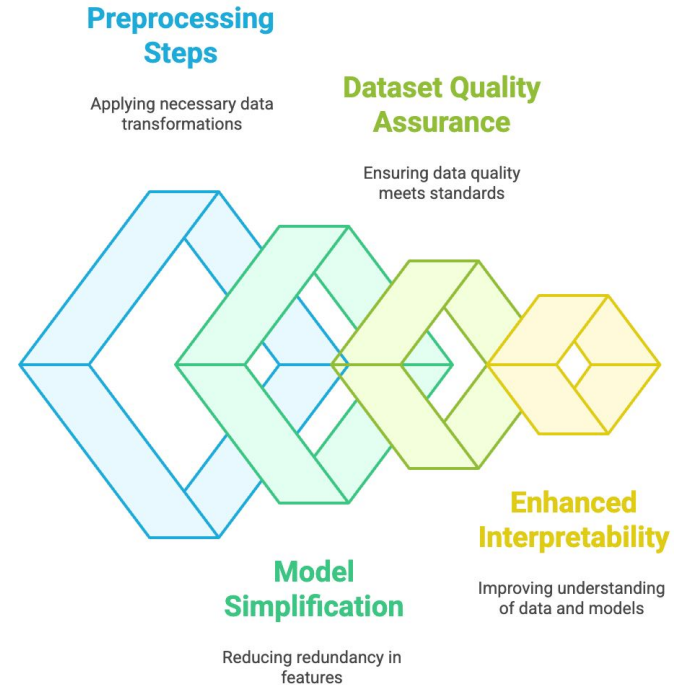
    # Update the annotation
    updated_annotations.append({
        "image_id": ann["image_id"],
        "category_id": ann["category_id"],
        "bbox": [x_min, y_min, x_max - x_min, y_max - y_min], # Save in COCO format
        "area": (x_max - x_min) * (y_max - y_min),
        "iscrowd": ann["iscrowd"],
    })
```

Statistical Analysis: Color Composition

We need to check for color channel correlations because high correlation means there is significant overlap in the intensity variations between channels (e.g., bright areas are bright across all channels). This redundancy may:

- Reduce the effective dimensionality of the data.
- Make some features less informative for distinguishing patterns.

Optimizing Data Pipeline through Color Channel Correlation





Statistical Analysis: Color Composition

The values represent the correlations between the intensity levels of the Red (R), Green (G), and Blue (B) channels across all images in the dataset.

1.0 on the diagonal indicates that each channel is perfectly correlated with itself (as expected).

High values (close to 1) off the diagonal (e.g., 0.92 between R and G, 0.84 between R and B) indicate that there is a strong linear relationship between the channels.

Average Color Channel Correlations:

```
[[1.          0.92134976 0.84394765]
 [0.92134976 1.          0.94482624]
 [0.84394765 0.94482624 1.          ]]
```

Average Shannon Entropies (R, G, B): [7.41554089 7.43381844 7.31448654]

Statistical Analysis: Color Composition

High correlation: Suggests that the images have a lot of **similar intensity variations across the channels**.

R > G > B correlation: The Red and Green channels have the strongest relationship (0.92), while the Blue channel is slightly less correlated with the other two. This is typical in **natural image datasets**, where Red and Green tend to dominate due to how humans and cameras perceive light.

Statistical Analysis: Color Composition

Shannon entropy measures the amount of uncertainty or information contained in the intensity values of a color channel. Higher entropy indicates more diverse and complex intensity distributions in the channel, while lower entropy implies simpler, less varied distributions.

Our findings suggest that:

- The Green channel contains the most diverse color information or detail in the images.
- The Blue channel is slightly less varied, which is common because many natural scenes have less variation in blue tones (e.g., skies, water, etc.).

Statistical Analysis: Color Composition

```
def color_composition_analysis(selected_images):
    correlations = []
    shannon_entropies = []

    for image_info in selected_images:
        # Load image
        image = imread(image_info['coco_url'])
        if image is not None:
            # Compute color channel correlations
            R, G, B = image[:, :, 0], image[:, :, 1], image[:, :, 2]
            correlation_matrix = np.corrcoef([R.flatten(), G.flatten(), B.flatten()])
            correlations.append(correlation_matrix)

            # Compute Shannon entropy for each channel
            entropy = lambda channel: -np.sum(np.histogram(channel, bins=256, density=True)[0] * np.log2(
                np.histogram(channel, bins=256, density=True)[0] + 1e-8))
            shannon_entropies.append([entropy(R), entropy(G), entropy(B)])

    avg_correlation = np.mean(correlations, axis=0)
    avg_entropy = np.mean(shannon_entropies, axis=0)

    print(f"Average Color Channel Correlations:\n{avg_correlation}")
    print(f"Average Shannon Entropies (R, G, B): {avg_entropy}")

color_composition_analysis(selected_images)
```



Statistical Analysis: Quality Metrics

```
import cv2

def quality_metrics_analysis(selected_images):
    sharpness_scores = []
    blurriness_scores = []

    for image_info in selected_images:
        # Load image
        image = imread(image_info['coco_url'])
        if image is not None:
            gray_image = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

            # Compute sharpness using Sobel gradients
            gradient_x = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)
            gradient_y = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ksize=3)
            sharpness_scores.append(np.sqrt(np.mean(gradient_x**2 + gradient_y**2)))

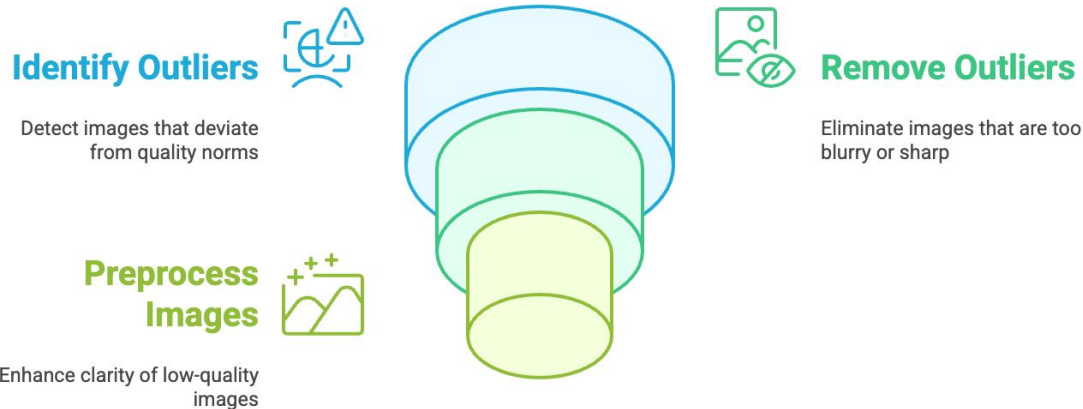
            # Compute blurriness using variance of Laplacian
            blurriness_scores.append(cv2.Laplacian(gray_image, cv2.CV_64F).var())

    print(f"Average Sharpness: {np.mean(sharpness_scores)}")
    print(f"Average Blurriness (Variance of Laplacian): {np.mean(blurriness_scores)}")

quality_metrics_analysis(selected_images)
```

Statistical Analysis: Quality Metrics

Sharpness is a measure of edge gradients or the contrast between adjacent pixels. Higher sharpness typically means clearer, more well-defined edges in the images. The sharpness value of 121.263 (typically range between 50 and 150) suggests that the dataset images generally have well-defined edges.



Statistical Analysis: Quality Metrics

Blurriness is often quantified using the variance of the Laplacian, where lower variance indicates less texture and sharper images, and higher values indicate more texture and detail. The blurriness score of 2972.625 (1000–2500) is relatively high, which implies that the dataset contains detailed images with a fair amount of texture.

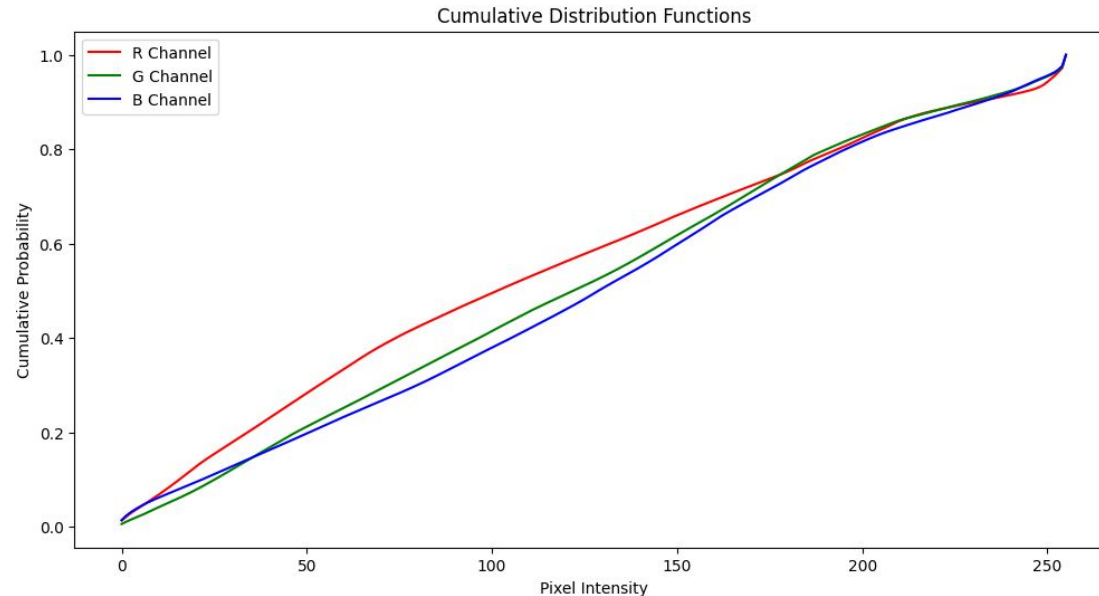


Visualization Analysis

1. **Visualizing Distributions**
 - a. **Cumulative Distribution Functions (CDFs):** Visualize cumulative pixel intensities.
2. **Spatial Features**
 - a. **Heatmaps:** Display average or variance heatmaps for spatial patterns.
 - b. **Edge Detection Visualizations:** Use Sobel or Canny filters to highlight edges.
3. **Sample Images**
 - a. **Representative Samples:** Show examples of typical, edge-case, and outlier images.
4. **Data Augmentation Effects**
 - a. **Augmentation Comparisons:** Visualize how transformations (e.g., rotation, flipping, cropping) affect images.

Visualization Analysis: Distributions

The CDF curves for the green and blue channels are close to each other, while the red channel shows noticeable deviation in lower intensities. This could indicate an imbalance in the dataset, with red being more dominant in certain parts of the image.



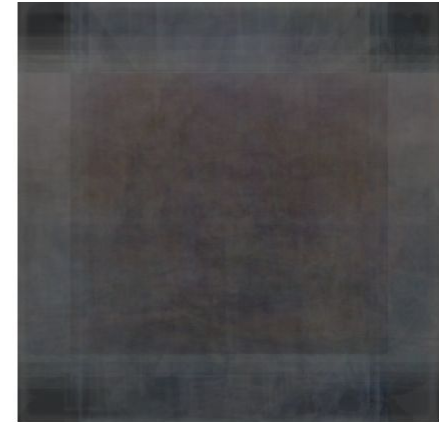
Visualization Analysis: Heatmaps

1. The heatmap represents the average pixel intensity at each spatial location across all images in the dataset. Darker areas indicate lower average intensity, while brighter areas indicate higher average intensity.
2. The heatmap represents the variance of pixel intensities at each spatial location across all images. Higher variance indicates greater variability in pixel intensity (e.g., areas with changing features or textures), while lower variance suggests more uniformity.

Average Intensity Heatmap



Variance Heatmap



Visualization Analysis: Edge Detection

Edge detection visualizations are performed to highlight the boundaries, shapes, and textures in images, enabling us to analyze the clarity and structural details of the dataset. This helps identify the quality of edges, assess image sharpness, and ensure sufficient feature richness for tasks like object detection or segmentation.

Original Image



Sobel Edge Detection



Canny Edge Detection



Visualization Analysis: Sample Images

Sample 0



Sample 1



Sample 2



Sample 3

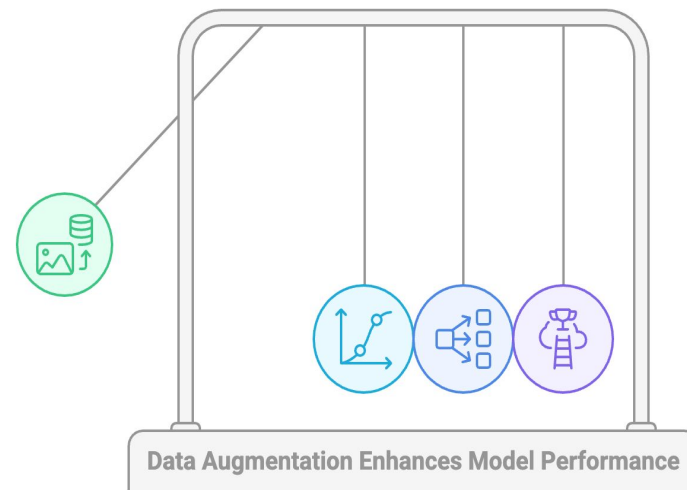


Sample 4



Visualization Analysis: Augmentation

Data augmentation is a technique used to artificially expand a dataset by applying transformations such as flipping, rotation, scaling, cropping, or color adjustments to the existing images.



**Data
Augmentation**

Expands dataset diversity

**Improved
Generalization**

Better model adaptability

**Reduced
Overfitting**

Less model bias

**Enhanced
Performance**

Higher accuracy rates

Visualization Analysis: Augmentation

```
def visualize_augmentations(image_path):  
    """Applies and displays data augmentations."""  
    img = Image.open(image_path)  
    aug_images = [  
        F.hflip(img),  
        F.vflip(img),  
        F.rotate(img, 45),  
        F.rotate(img, 90),  
        F.resize(img, (256, 256))  
    ]
```

Augmentation 1



Augmentation 2



Augmentation 3



Augmentation 4



Augmentation 5



Dimensionality Analysis

1. Compression and Feature Space

Singular Value Decomposition (SVD): Analyze rank and compression of image matrices.

2. Complexity Estimation

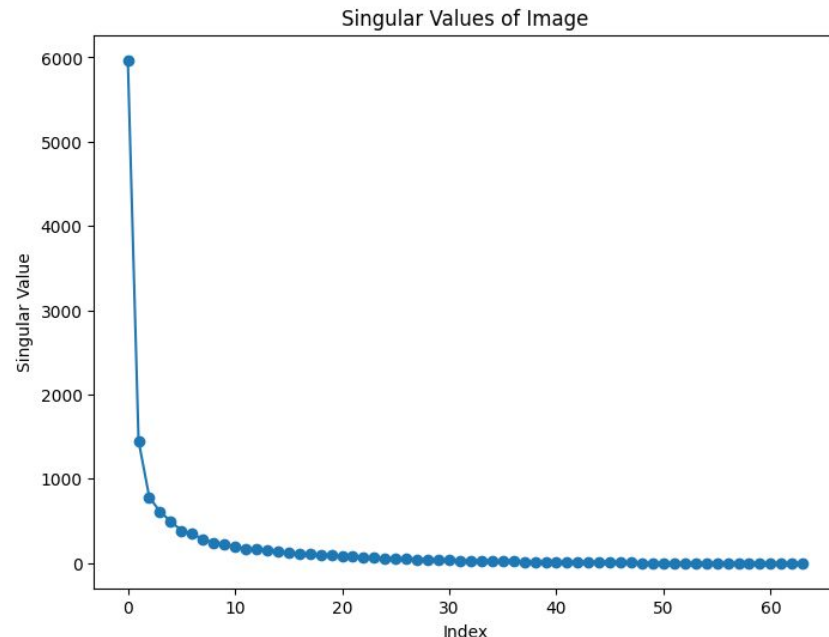
Fractal Dimension: Measure the complexity of image structures.

Dimensionality Analysis: SVD

SVD identifies the "important" components of an image by ranking the singular values (S), which represent the amount of variance explained by each singular vector.

Singular values represent the amount of information or variance captured by each component of the image.

- The image has a lot of redundancy or low-rank structure.
- A significant portion of the image's information can be retained using only the first few singular values.
- The remaining singular values (those close to zero) contribute little to the image's structure and likely represent noise or minor details.

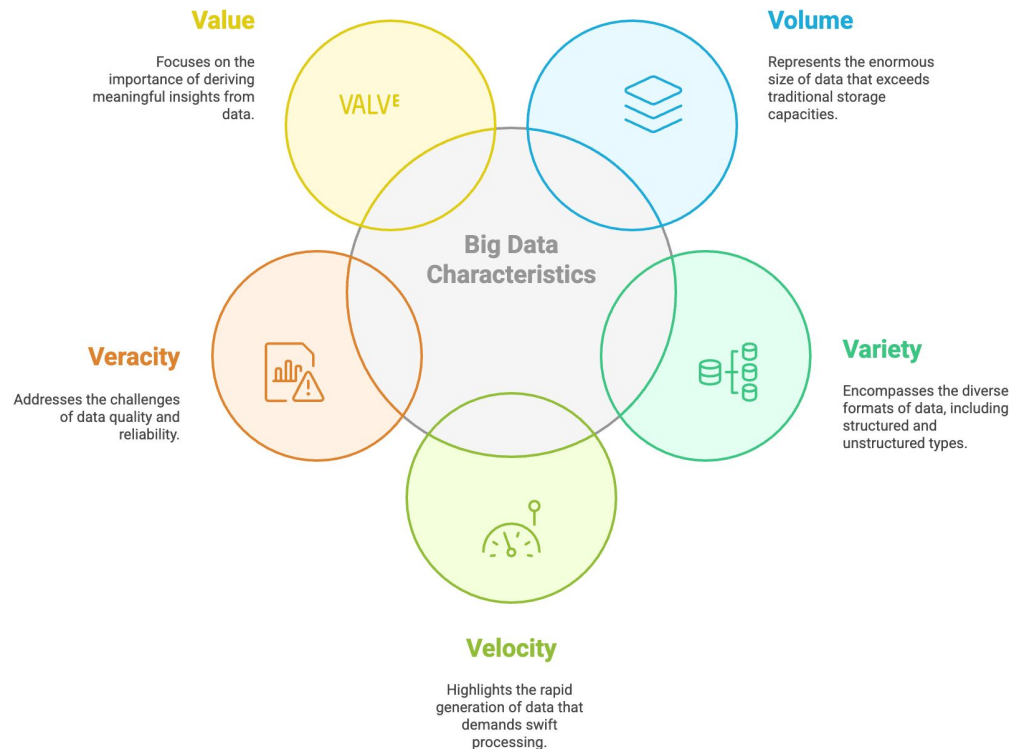


Exploring Big Data

What is Big Data?

Big data refers to the vast volumes of structured and unstructured data that are generated every second from various sources.

Understanding the Five V's of Big Data Characteristics

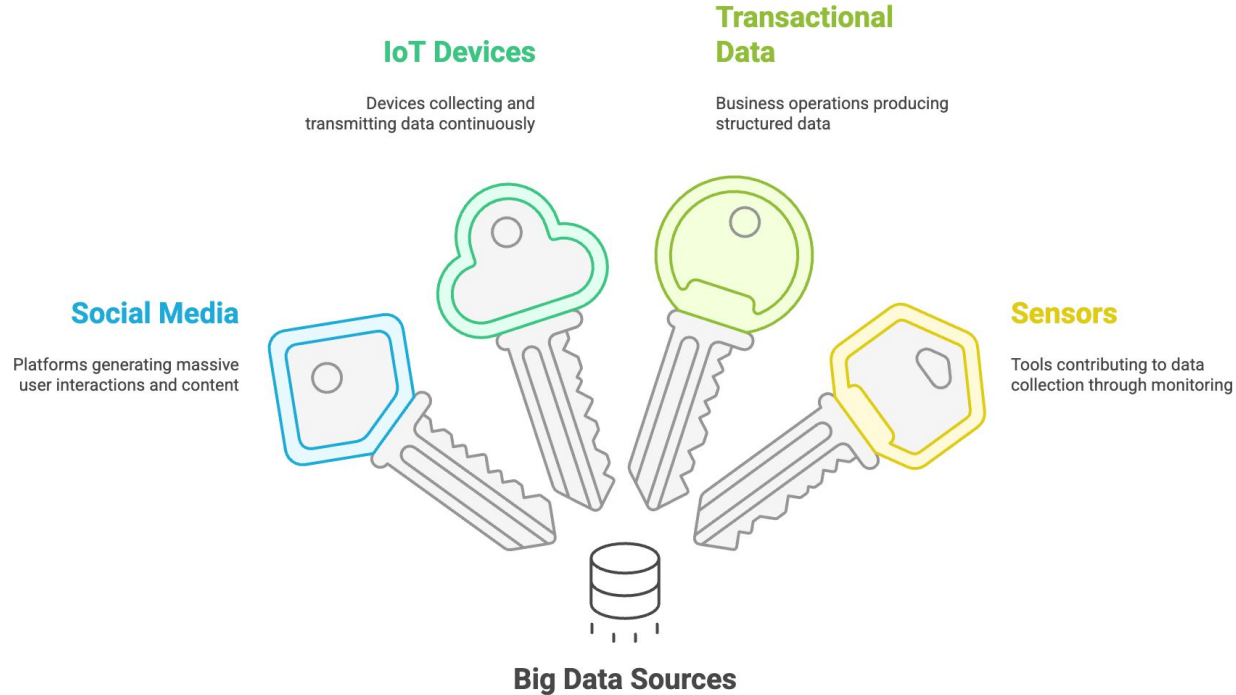


Sources of Big Data

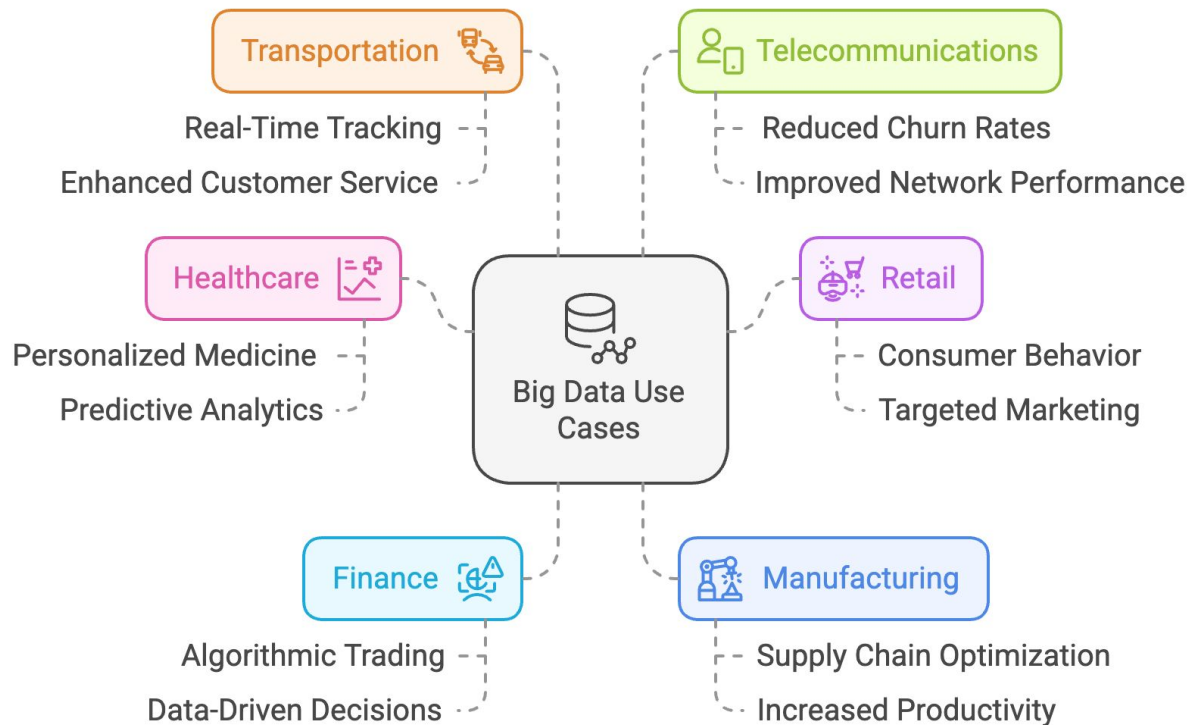


AMERICAN
UNIVERSITY
OF BEIRUT

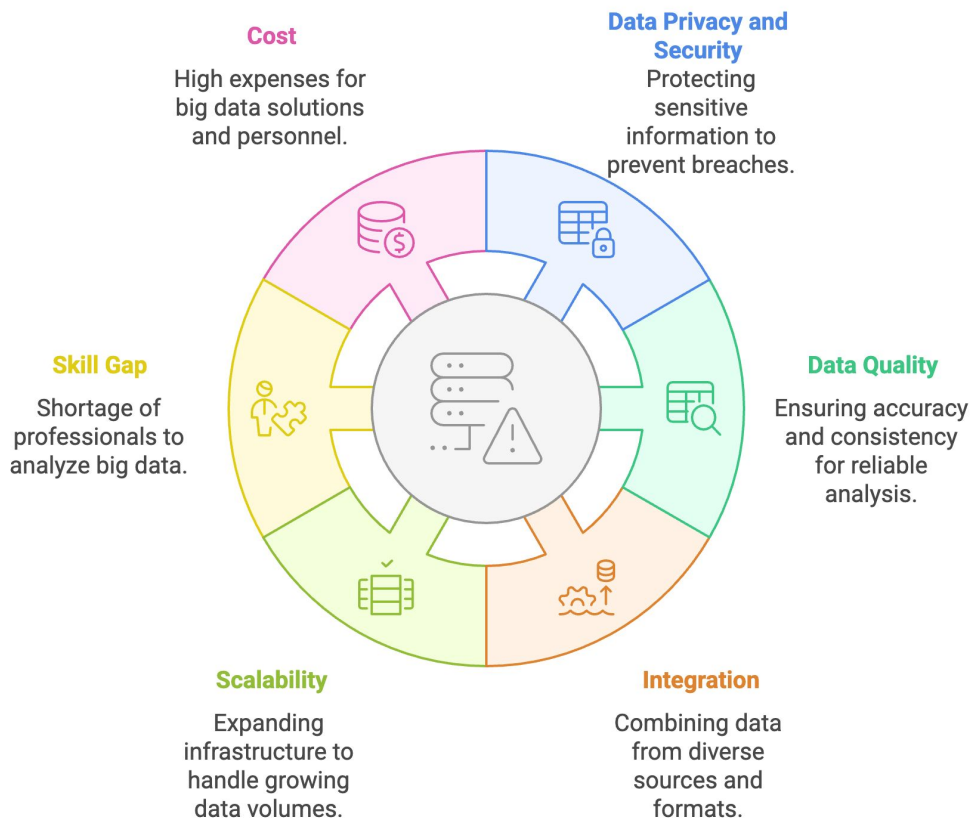
Diverse Origins of Big Data in the Digital Age



Big Data Use Cases

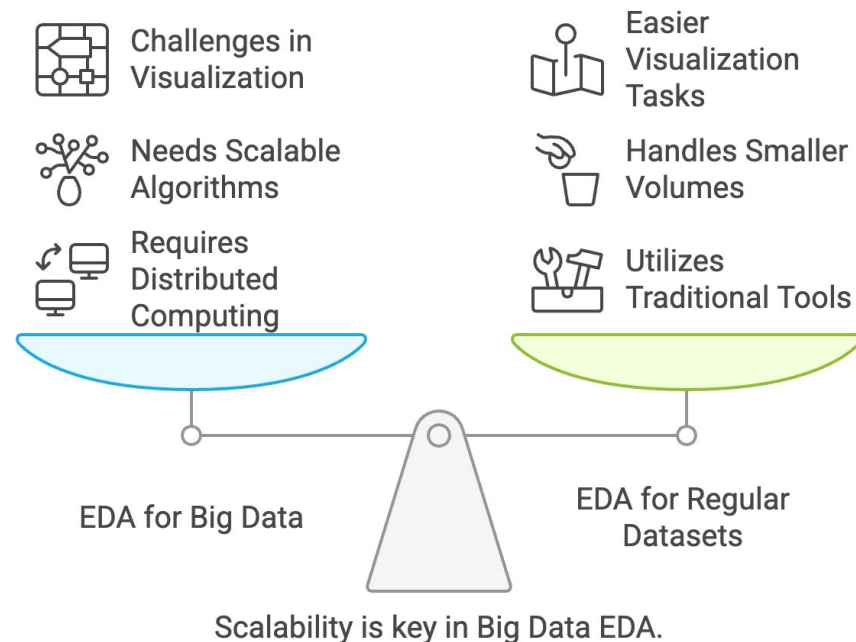


Challenges of Big Data



EDA On Big Data

EDA for big data shares many core principles with EDA for regular-sized datasets, such as identifying data distributions, handling missing values, detecting outliers, and exploring feature relationships. However, the key difference lies in the **need for scalability**. With big data, traditional tools and methods may falter due to memory constraints, **high computational demands**, and the **sheer volume of data**.





Thank you